B4-2
장애 분석 통합 교안

명령을 입력하는 이유부터
출력 해석과 평가 설명까지

OOM Crash · CPU Latency · Deadlock

미션 원문 대조 · 단계별 실습 · 증거 분석 · 평가 질문 · 용어집

이 책의 학습 목표

왜 이 명령을 썼는지, 출력이 무엇을 뜻하는지, 어떤 증거로 원인을 판단했는지 스스로 설명합니다.

대상 저장소

github.com/bidulgiya999/codyssey_work_B4-2

학습 자료 작성: 2026-10-23
수치의 원본 실험: 2026-10-09 KST
새 재실습의 결과·PID는 직접 기록합니다.

차례

학습 안내: 이 책을 읽는 순서 3

출력은 왜 이런 모양으로 나오는가 5

실험 설정값은 왜 이렇게 골랐는가 5

제1부. 명령의 이유와 출력 해석 8

1. Ubuntu에 접속하고 실행 계정 확인 . . . 8
2. 파일 선택, 작업 폴더, 실행 권한 . . . 8
3. 환경변수와 필수 파일 준비 . . . 9
4. 실행별 증거와 PID 관계 . . . 10
5. monitor.sh를 설명할 수 있게 읽기 . . . 11
6. 메모리: 전체 RAM과 앱 한도 구분 . . . 12
7. 메모리 설정 비교와 임시 조치 . . . 13
8. CPU: 세 종류의 숫자 구분 . . . 14
9. CPU 종료와 설정 변경 검증 . . . 15
10. Deadlock: 살아 있음과 진행함 구분 . . . 15
11. Deadlock 회피, 수동 종료와 종료 코드 . . . 16
12. 보고서를 증거와 함께 설명하기 . . . 17

제2부. 준비부터 여섯 번의 비교 실험까지 19

실습 절차 1. 자료와 관제 파일 준비 . . . 19
실습 절차 2. 공통 수집 함수를 이해하고 정의 . . . 20
실습 절차 3. 메모리 두 번 실행하고 비교 . . . 23
실습 절차 4. CPU 두 번 실행하고 비교 . . . 24
실습 절차 5. 교착 재현과 회피 비교 . . . 25
실습 절차 6. 보고·백업·설명으로 마무리 . . . 26

제3부. 평가 질문과 증거를 연결하는 설명 28

제4부. 찾아보는 용어집 33

제5부. 증거 링크별 출력 해설 39

학습 안내: 이 책을 읽는 순서

작성일: 2026-10-23. 대상 저장소는 bidulgiya999/codyssey_work_B4-2입니다. 새로 첨부한 미션 원문과 Downloads/agent-app-leak의 제공 파일 목록을 확인해 작성했습니다. 제공 파일은 agent-leak-app-x86과 agent-leak-app-arm64입니다. 디컴파일·리버스 엔지니어링은 하지 않았습니다.

이 과제의 목적은 프로그램을 다시 켜는 데서 끝나지 않고, **왜 중단되거나 멈췄는지 증거로 설명하는 것**입니다. SSH·방화벽·cron 설정을 구축하는 다른 미션과 구분합니다.

읽는 순서는 **전체 흐름 → 기초 명령과 개념 → 실제 실습 순서 → 보고서 작성 → 평가 설명 → 용어집**입니다. 입력 명령은 코드 블록에 표시했고 예전 출력은 따로 표시했습니다. 출력 예시를 명령으로 입력하지 않습니다. PDF의 긴 코드가 시각적으로 줄바꿈되면 실행용 복사는 GitHub Markdown의 원문을 사용합니다.

모든 단계에 적용할 여섯 가지 질문

1. 지금 무엇을 확인하려는가?
2. 왜 이 명령과 옵션을 선택했는가?
3. 실행하면 어떤 일이 일어나며 무엇은 바뀌지 않는가?
4. 실제 출력의 값·단위·상태·시각은 무슨 뜻인가?
5. 이 결과로 무엇까지 판단할 수 있고, 무엇은 더 확인해야 하는가?
6. 평가자에게 어떤 원본 파일을 열어 설명할 것인가?

처음부터 끝까지의 실행 순서

단계	수행 내용	이 순서인 이유
1	Ubuntu·계정·CPU 종류 확인	실행 조건과 맞는 바이너리 결정
2	새 작업 폴더와 실행 권한 준비	원본을 보존하고 일반 계정 실행 가능 여부 확인
3	환경변수·폴더·키 준비	부팅 실패와 장애 실험을 혼동하지 않기 위해
4	monitor.sh 준비·문법 검사	짧은 장애가 나기 전에 관제를 준비
5	실행별 설정과 증거 폴더 생성	비교 자료의 조건·시각을 추적
6	앱과 관제 함께 시작	실행 직후와 종료 전 수치를 놓치지 않기 위해
7	OOM 128과 256 비교	메모리 한도만 바꿔 생존 시간 확인
8	CPU 80과 40 비교	부하·냉각·보호 종료·생존 여부 확인
9	Deadlock true와 false 비교	종료되지 않은 상태의 정체와 진행을 구분
10	수동 종료 기록·프로세스 정리	자동 종료와 실험자 조치를 구분하고 포트 반환
11	증거 검토·보고서 3건 작성	결론을 로그·명령 출력과 연결
12	구두 설명과 질문 연습	본인이 수행·판단·한계를 설명할 수 있는지 확인

첨부 미션과 교안의 대응

과제 요구	수집할 증거	학습 내용
일반 사용자와 필수 환경	UID, printenv, 폴더·키 권한, Boot Sequence	환경 준비
OOM 물리 메모리 증가	monitor CSV의 RSS	메모리 지표와 임계치
OOM 종료와 최소 2회 비교	MemoryGuard 로그, 128/256 설정, 생존 시간	임시 조치의 효과와 한계
특정 프로세스 CPU 상승	top의 PID별 사용률과 관측 기간	평균·구간·내부 지표 구분
Watchdog 종료와 설정 비교	SIGTERM 로그, 80/40 비교	보호 종료와 냉각 검증
PID가 남은 교착 상태	ps, ps -L, 로그 크기·수정 시각	살아 있음과 진행함 구분
WAITING/BLOCKED와 락 관계	자원 획득·추가 요청 로그	교착 4대 조건과 순환 대기
동시 설정 변경	true/false 새 실행의 스냅샷	교착 경로 회피
보고서 3건과 공통 템플릿	현상·증거·원인·조치/검증	기술 보고와 평가

첨부 원문의 10분·30분·MEM 96%와 Round-Robin 로그는 **참고 예시**입니다. 본 저장소의 실제 수치처럼 옮기지 않습니다. 종료 로그의 문구도 의미가 일치하는 실제 로그를 제시하며, 예시 문장을 만들거나 바꾸지 않습니다.

예전 기록과 이번 학습의 경계

이 책의 실제 수치는 GitHub에 보존된 **2026-10-09 실험** 기준입니다. 10월 13일의 재실습 수치와 섞지 않습니다. 새로 실행하면 새 폴더·새 PID·실제 시각을 기록합니다. 교안 작성 과정에서 제공 바이너리를 새로 실행해 결과를 추가한 것은 아닙니다.

합격이나 점수를 보장하는 자료는 아닙니다. 모르는 내용과 확인하지 못한 구현을 정확하게 구분해 설명하는 것이 목표입니다.

출력은 왜 이런 모양으로 나오는가

보이는 현상	가능한 뜻	다음 확인
아무 출력 없이 프롬프트 복귀	cp·chmod·mkdir 등이 정상 완료했을 수 있음	ls·권한·실제 파일로 검증
ps·ss에 제목만 표시	조회 조건에 맞는 행이 없음	조회 대상·필터·시각이 맞는지 확인
숫자 하나만 출력	id·wc 등 특정 값을 요청했기 때문	숫자의 단위·정의를 명령과 함께 읽기
[1] 1234	백그라운드 작업 번호와 PID	앱인지 기록 도구인지 구분
Done	해당 백그라운드 작업이 끝남	앱·관제 중 어느 작업인지 확인
Killed·Exit 143	셸이 종료를 알림	앱 로그·종료 코드·수동 기록 대조
Is a directory	디렉터리 경로를 실행하려 함	이동은 cd, 조회는 ls 사용
command not found	명령을 찾지 못함	출력 예시를 입력했는지·오타·설치 확인
Permission denied	실행·접근 권한 등에 문제	소유자·권한·상위 경로 확인
로그가 화면에 안 보임	파일로 출력했거나 버퍼링·실제 정체 가능	tail, PID, 파일 크기, 시각으로 구분

조회 오류가 있었는데 단순히 출력이 없었다는 이유만으로 정상·종료를 단정하지 않습니다.

실험 설정값은 왜 이렇게 골랐는가

이 값들은 과제의 허용 범위 안에서 **장애를 관찰하고 조건을 비교하려고 선택한 실험 설정**입니다. 모든 서버의 권장 운영값이나 Windows 전체 자원을 제한하는 값이 아닙니다. 바이너리 내부 코드는 확인하지 않았으므로 설정별 동작의 설명은 부팅·작업·보호 로그로 확인한 범위에 한정합니다.

실험	MEMORY_LIMIT	CPU_MAX_OCCUPY	MULTI_THREAD_ENABLE	이 조합을 사용한 이유
OOM Before	128	100	false	낮은 메모리 한도에서 증가와 보호 종료를 짧게 관측. 동시 트랜잭션 교착 경로를 끔
OOM After	256	100	false	메모리 한도만 2배로 바꿔 생존 시간 비교. 다른 두 설정은 고정
CPU Before	512	80	false	높은 메모리 한도를 유지하고 CPU 부하 상승·보호 종료를 관찰. 교착 경로를 끔
CPU After	512	40	false	CPU 설정만 낮춰 부하 냉각·생존을 비교
Deadlock Before	512	40	true	앞서 진행을 확인한 메모리·CPU 조합을 고정하고 동시 트랜잭션 경로 활성화

실험	MEMORY_LIMIT	CPU_MAX_OCCUPY	MULTI_THREAD_ENABLE	이 조합을 사용한 이유
Deadlock After	512	40	false	동시 설정만 끄고 로그·작업의 진행 재개를 비교

MEMORY_LIMIT: 왜 128, 256, 512인가

과제의 허용 범위는 정수 **50~512MB**입니다. 128은 메모리가 누적될 때 비교적 짧게 한도에 도달하도록 고른 값입니다. 로그는 약 3초마다 25MB씩 증가했고, 125 다음 150MB에서 150 >= 128을 기록했습니다. 그래서 128에서 정확히 멈추는 출력이 아닙니다.

256은 128의 두 배이므로 한도 상향의 효과를 설명하기 쉽습니다. 다만 시작 메모리, 할당 간격, 한도를 넘는 단계가 있으므로 생존 시간이 정확히 두 배가 되는 것은 아닙니다. 실제로 17.202초에서 32.363초로 약 1.88배였습니다. 250 다음 275MB에서 한도를 넘었습니다. 로그의 Recommend Over 256MB는 **256보다 큼**을 뜻하므로 256 자체도 경고가 남습니다.

512는 과제에서 허용하는 최대값이며 CPU·교착 비교에서 메모리 설정을 충분히 높게 고정하기 위해 골랐습니다. 512를 설정한다고 노트북 RAM을 즉시 512MB 예약하거나 모든 실행의 RSS를 512MB 이하로 강제 제한하는 것은 아닙니다. 실제 로그 기준 보호·회수 동작과 OS 상주량은 별도로 확인해야 합니다. 정상 진행 경로에서 별도 메모리 회수 로그도 관측됐으므로, 단순히 낮은 한도 실험의 누적 시간을 길게 늘린 것이라고 내부 구현까지 단정하지 않습니다.

왜 50이나 200은 안 쓰나? 허용 범위 안이므로 사용할 수 있습니다. 이번에는 128/256의 단순 비교와 512의 상한 고정을 택했습니다. 다른 값이 틀린 것은 아니며 새 값을 쓰면 새 결과를 수집해야 합니다. 앱 한도를 낮춘 이유와 Windows PC Manager의 메모리 비율은 별개입니다.

CPU_MAX_OCCUPY: 왜 100, 80, 40인가

과제의 허용 범위는 정수 **10~100%**입니다. 이 환경변수는 앱에 전달하는 설정이며, 운영체제의 CPU quota나 코어 수 제한을 설정하는 명령이 아닙니다.

OOM 실험의 100은 메모리 비교 동안 CPU 설정을 높은 값으로 동일하게 유지한 선택입니다. "CPU 보호를 비활성화한다"는 뜻은 확인하지 않았습니다. 해당 실행에서는 MemoryWorker와 MemoryGuard 로그가 먼저 관측됐다는 사실로 설명합니다. 높은 값을 모든 실험에 사용하면 CPU 장애나 교착을 해결한다는 뜻도 아닙니다.

CPU Before의 80은 부팅이 허용하는 범위 안에서 부하 상승과 보호 종료를 관찰하려고 택했습니다. 로그의 Recommend Under 50%는 **50보다 작게 권장**한다는 뜻입니다. 80은 이 권고보다 높지만 입력 범위 오류는 아닙니다. 실제로 앱 내부 53.11%에서 Watchdog가 SIGTERM을 기록했으므로 **80이 정확한 종료 임계치라고 설명할 수 없습니다.** 설정 처리·측정 기준·내부 보호 로직의 상세 구현은 확인하지 않았습니다.

After의 40은 권고인 50 미만에 들어가는 값입니다. 80과 다른 조건은 고정하고, 앱의 40% 도달 후 냉각 로그와 top 최대 40.0%, 관측 시점 PID 유지를 함께 확인했습니다. "40이 유일한 정답"이나 "노트북 전체 CPU를40%로 제한"이라는 뜻은 아닙니다. 실제 관측값을 근거로 선택 효과를 설명합니다.

왜 50은 안 쓰나? 앱 문구가 50 미만을 권장하므로 비교용으로 40을 선택했습니다. 50이 어떤 동작을 하는지는 별도 실험 없이 보장하지 않습니다. 40이라는 설정값만으로 응답 지연 개선이나 영구 안정성을 입증하지도 않습니다.

MULTI_THREAD_ENABLE: 왜 true와 false인가

OOM·CPU 비교에서는 false로 고정해 동시 트랜잭션 교착 경로의 영향을 줄였습니다. 교착 비교에서는 다른 두 설정을 512/40으로 고정하고 true에서 false만 바꿨습니다. true에서는 서로 반대 순서로 자원을 보유·요청하는 로그와 정체가 있었고, false에서는 관측 구간에 로그와 작업이 진행했습니다.

이 설정의 false는 프로세스 전체에 스레드가 하나뿐이라는 뜻이 아닙니다. 실제 After에도 스레드3개가 관측됐습니다. 설명할 때는 "이번에 관찰한 동시 트랜잭션의 교착 경로를 회피했다"라고 범위를 정합니다.

이 구성의 용어 설명과 평가 답변

허용 범위는 앱이 입력으로 받아들이는 조건이고, 권장값은 로그가 안내하는 운영상 권고이며, 실험값은 비교를 위해 선택한 값입니다. 임계치는 실제 보호 동작의 판정 기준입니다. 네 개념을 같은 것으로 보지 않습니다. 통제 변수는 고정한 조건, 독립 변수는 의도적으로 바꾼 조건, 결과 지표는 생존 시간·RSS·CPU·로그 진행 등입니다.

평가 답변 예: "128과256은 메모리 한도만 바꾼 비교를 위해 선택했습니다. 한도를 높여 생존 시간은 늘었지만 두 실행 모두 MemoryGuard로 종료돼 임시 조치였습니다. CPU의80과40은 허용 범위 안에서 권고50 미만 여부와 보호/냉각 동작을 비교한 값이며, 설정80을 실제 종료 기준80%로 단정하지 않았습니다."

제1부. 명령의 이유와 출력 해석

1. Ubuntu에 접속하고 실행 계정 확인

입력할 명령 - Windows PowerShell

```
wsl -d Ubuntu
```

Ubuntu를 시작하는 명령입니다. Ubuntu가 이미 설치돼 있다면 재설치할 필요는 없습니다. PS C:₩...>는 PowerShell, jae0608@HP:~$는 Ubuntu 셸의 프롬프트입니다. 프롬프트 자체는 입력하지 않습니다. Microsoft WSL 명령 문서

입력할 명령 - Ubuntu

```
cd ~
pwd
whoami
id -u
uname -r
uname -m
```

명령	왜 사용하는가	예전 실제 결과와 의미
cd ~	사용자 홈으로 이동	성공 시 보통 출력 없음
pwd	현재 위치 확인	홈 경로 /home/jae0608
whoami	실행 계정 확인	jae0608
id -u	root인지 확인	1000, root의 UID 0이 아님
uname -r	커널 확인	6.18.40.1-microsoft-standard-WSL2
uname -m	CPU 아키텍처 확인	x86_64

판단: 과제의 일반 사용자 실행 조건을 확인하고 파일 선택의 기준을 확보했습니다. 새 실행 결과가 다르면 그 값을 기록합니다. 환경 원본

설명 연습: "실행 전에 계정과 CPU 종류를 확인한 이유는 무엇인가?"

이 단계의 용어 설명

용어	이 단계에서의 뜻과 읽는 방법
WSL2	Windows에서 Linux 커널을 사용하는 실행 환경입니다. PowerShell과 Ubuntu 셸의 명령을 구분합니다.
UID	사용자의 숫자 식별자입니다. id -u의 1000은 계정 번호이며 사용률이 아닙니다.
커널·아키텍처	커널은 자원을 관리하는 OS 핵심이고 x86_64는 실행 파일을 선택하는 CPU 명령 체계입니다.

2. 파일 선택, 작업 폴더, 실행 권한

입력할 명령 - Ubuntu, 파일 형식 조회

```
file /mnt/c/Users/1234v/Downloads/agent-app-leak/agent-leak-app-*
```

file은 파일 형식을 조회하며 디컴파일 명령이 아닙니다. ELF 64-bit ... x86-64는 Linux x86-64용, ARM aarch64는 ARM용입니다. 예전 환경은 x86_64여서 x86 파일을 선택했습니다. 파일 이름만이 아니라 실제 형식과 컴퓨터 아키텍처를 비교합니다.

새 학습 경로는 과거 ~/jae-test와 구분한 ~/agent-b42-study를 사용합니다. 아래 복사는 **아키텍처가 x86_64임을 확인한 경우**입니다.

입력할 명령 - Ubuntu

```
mkdir -p ~/agent-b42-study
cp /mnt/c/Users/1234v/Downloads/agent-app-leak/agent-leak-app-x86 ~/agent-b42-study/agent-leak-app
chmod u+x ~/agent-b42-study/agent-leak-app
ls -lh ~/agent-b42-study/agent-leak-app
```

구문	목적
/mnt/c	Ubuntu에서 Windows C드라이브에 접근
mkdir -p	필요한 폴더 생성, 기존 폴더는 유지
cp 원본 대상	원본을 보존하고 실습 복사본 준비
chmod u+x	소유자에게 실행 권한 추가
ls -lh	권한·소유자·크기를 확인

출력 읽기: -rwxr-xr-x의 첫 -는 일반 파일, 다음 rwx는 소유자의 읽기·쓰기·실행 권한입니다. 실행 권한이 있다고 부팅 조건까지 통과한 것은 아닙니다.

자주 생기는 혼동: pwd가 출력한 경로를 그대로 입력하면 Bash는 그 경로를 실행하려 합니다. 폴더라면 Is a directory가 나올 수 있습니다. 이동은 cd 경로, 내용 조회는 ls 경로입니다.

이 단계의 용어 설명

용어	이 단계에서의 뜻과 읽는 방법
ELF	Linux 실행 파일 형식입니다. file 출력의 x86-64 또는 aarch64와 uname -m을 대조합니다.
경로·홈	파일 위치를 나타냅니다. ~는 현재 사용자의 홈이며 /mnt/c는 Windows C드라이브 접근 경로입니다.
권한 r/w/x	읽기·쓰기·실행 권한입니다. chmod u+x는 소유자에게 실행 권한을 추가합니다.

3. 환경변수와 필수 파일 준비

환경변수는 프로그램에 전달하는 설정입니다. 파일 저장, 현재 셸 적용, 자식 프로세스 전달을 구분합니다. 저장소의 예전 env.sh는 ~/jae-test 경로이므로 새 학습에서는 다음 파일을 별도로 만듭니다.

입력할 명령 - Ubuntu

```
cat > ~/agent-b42-study/env.sh <<'EOF'
export AGENT_HOME="$HOME/agent-b42-study"
export AGENT_PORT=15034
export AGENT_UPLOAD_DIR="$AGENT_HOME/upload_files"
export AGENT_KEY_PATH="$AGENT_HOME/api_keys"
export AGENT_LOG_DIR="$AGENT_HOME/logs"
export MEMORY_LIMIT=128
export CPU_MAX_OCCUPY=100
```

```
export MULTI_THREAD_ENABLE=false
EOF
source ~/agent-b42-study/env.sh
mkdir -p "$AGENT_UPLOAD_DIR" "$AGENT_KEY_PATH" "$AGENT_LOG_DIR"
printf '%s' 'agent_api_key_test' > "$AGENT_KEY_PATH/secret.key"
chmod 600 "$AGENT_KEY_PATH/secret.key"
```

구문	왜 사용하는가
<<'EOF'	파일을 쓰는 시점에는 $HOME 등을 그대로 저장
source	설정을 현재 셸에서 실행
export	이후 실행할 자식 프로그램에 변수 전달
printf '%s'	지정 문자열을 추가 줄바꿈 없이 기록
>	파일 생성 또는 기존 내용 덮어쓰기
chmod 600	소유자만 읽기·쓰기 허용

입력할 명령 - 적용 결과 조회

```
printenv AGENT_HOME AGENT_PORT AGENT_UPLOAD_DIR AGENT_KEY_PATH ₩
  AGENT_LOG_DIR MEMORY_LIMIT CPU_MAX_OCCUPY MULTI_THREAD_ENABLE
ls -ld "$AGENT_UPLOAD_DIR" "$AGENT_KEY_PATH" "$AGENT_LOG_DIR"
ls -l "$AGENT_KEY_PATH/secret.key"
wc -c < "$AGENT_KEY_PATH/secret.key"
ss -ltn 'sport = :15034'
```

키 파일은 지정 문자열이면 18바이트입니다. 크기만으로 내용 일치까지 증명할 수는 없습니다. ss는 대기 중인 TCP 소켓을 조회하며 제목만 나오면 조회 시점에 해당 포트의 리스너가 없다는 뜻입니다. 실제 바인딩 성공은 앱 실행으로 확인합니다.

앱은 일반 사용자, 환경변수, 필수 파일, 포트, 로그 권한, 미션 환경의 **6단계 부팅 검사**를 수행했습니다. All Boot Checks Passed! / Agent READY와 이후 작업 로그를 함께 확인합니다. 권장값 WARNING은 부팅 실패와 구분합니다.

설명 연습: "env.sh를 저장한 것과 source한 것은 어떻게 다른가? MEMORY_LIMIT은 Windows 전체 RAM 제한인가?"

이 단계의 용어 설명

용어	이 단계에서의 뜻과 읽는 방법
환경변수·export	프로그램에 전달하는 설정값입니다. export로 이후 시작하는 자식에게 전달합니다.
source	파일의 명령을 현재 셸에서 읽습니다. 새 터미널에서는 다시 적용해야 합니다.
리다이렉션·here-document	>는 출력을 파일로 보내고 <<'EOF'는 끝 표시까지의 내용을 변수 확장 없이 저장합니다.
포트·바인딩	서비스가 사용할 통신 번호와 주소를 확보하는 동작입니다. ss는 현재 리스닝 상태를 조회합니다.

4. 실행별 증거와 PID 관계

입력할 명령 - 첫 OOM 실행의 기록 위치 준비

```
RUN_DIR="$AGENT_HOME/evidence/oom-128-$(date +%Y%m%d-%H%M%S)"
mkdir -p "$RUN_DIR"
printf 'MEMORY_LIMIT=%s₩nCPU_MAX_OCCUPY=%s₩nMULTI_THREAD_ENABLE=%s₩n' ₩
  "$MEMORY_LIMIT" "$CPU_MAX_OCCUPY" "$MULTI_THREAD_ENABLE" > "$RUN_DIR/settings.txt"
```

AGENT_HOME은 실습 기준 경로, RUN_DIR은 실행 한 번의 증거 경로입니다. 새 폴더와 settings.txt가 실험 간 혼동을 줄입니다. 폴더 이름의 시각은 폴더 생성 시각이며 실제 앱 시작 시각과 다를 수 있습니다.

다음은 **실행 구문 해설**입니다. 실제 실행은 관제 준비와 부모·자식 PID 수집을 함께 진행합니다. 아래 두 줄만 실행하고 오래 기다리면 종료 전 관측을 놓칠 수 있습니다.

```
./agent-leak-app > "$RUN_DIR/app.log" 2>&1 &
APP_PID=$!
```

구문	의미
./	현재 폴더의 실행 파일
> app.log	표준 출력 저장
2>&1	오류 출력도 현재 표준 출력과 같은 곳으로 연결
&	백그라운드 실행
$!	직전 백그라운드 작업의 PID

부모만 관측하면 실제 작업 자식의 메모리 증가를 놓칠 수 있습니다. 실행 중에는 pgrep -P "$APP_PID" -x agent-leak-app으로 자식을 찾고 ps의 PPID로 관계를 검증합니다. PID 생성 전에 조회할 수도 있으므로 짧은 재시도가 필요할 수 있습니다.

PTY 실행에서는 $!가 script의 PID이므로 SESSION_PID로 저장하고 앱 PID는 별도로 찾습니다. 재실습에서는 예전 숫자 PID를 복사해 사용하지 않습니다.

이 단계의 용어 설명

용어	이 단계에서의 뜻과 읽는 방법
PID·PPID	프로세스 번호와 부모 번호입니다. 실행기와 실제 작업 자식을 구분해 관제합니다.
$!·백그라운드	&로 시작한 직전 작업의 PID를 $!로 얻습니다. [1]은 셸의 작업 번호입니다.
PTY·script	가상 터미널과 그 출력을 기록하는 도구입니다. 세션 PID와 앱 PID는 다를 수 있습니다.

5. monitor.sh를 설명할 수 있게 읽기

사용한 소스는 scripts/monitor.sh입니다. 현재 폴더에 소스를 저장한 뒤 chmod u+x monitor.sh로 실행 권한을 주고 bash -n monitor.sh로 문법을 검사했습니다. 문법 통과는 실제 관제 성공까지 의미하지 않습니다.

관제 실행 구문은 ./monitor.sh "$WORKER_PID" > "$RUN_DIR/monitor-$WORKER_PID.csv" 2> "$RUN_DIR/monitor-$WORKER_PID.log" &입니다. 이때 WORKER_PID는 현재 살아 있는 작업 자식의 값이어야 합니다.

소스의 부분	역할
LC_ALL=C	출력 형식 통일
$#, $1 검사	인자가 하나이며 양의 정수 형식인지 확인
ps -p ... -o ...	해당 PID의 지정 필드 조회
필드 뒤 =	ps 제목을 숨김
awk	열을 쉼표로 연결하고 상태 Z 행 제외

소스의 부분	역할
[[-n "$row"]] || break	유효 조회 행이 없으면 반복 종료
date -Iseconds	실제 수집 시각 추가
sleep 1	조회 사이에 1초 대기
>&2	안내 메시지와 CSV 데이터를 분리

CSV 열	읽는 방법
pid / ppid	프로세스와 부모 식별 번호
elapsed_s	프로세스 시작 후 경과 초
cpu_avg_pct	ps의 실행 이후 평균 CPU
mem_pct	Linux 전체 물리 메모리 대비 RSS 비율
rss_kib	물리 메모리에 상주하는 양, KiB
threads	스레드 수
state	R 실행 가능, S 대기, Z 좀비 및 부가 표시

조회·파일 출력에도 시간이 걸려 정확히 매 1.000초는 아닙니다. Z는 실행이 끝났지만 종료 상태가 회수되지 않아 정보가 남은 상태여서 제외합니다. 안내에 "no longer running"이 나오더라도 조회 실패 등도 가능하므로 앱 로그와 별도 ps 결과로 종료를 확인합니다. ps 매뉴얼

이 단계의 용어 설명

용어	이 단계에서의 뜻과 읽는 방법
CSV·표본	쉼표로 구분한 표와 한 시점의 측정값입니다. 1초 수집은 두 표본 사이 순간값을 놓칠 수 있습니다.
RSS·KiB	RAM에 상주하는 프로세스 메모리와 1024바이트 단위입니다. rss_kib를 1024로 나누면 MiB입니다.
NLWP·STAT	스레드 수와 프로세스 상태입니다. S는 수면, R은 실행/실행 가능, Z는 좀비 상태입니다.
LC_ALL=C	도구의 지역화 형식을 통일해 소수점과 출력 파싱을 안정시키는 설정입니다.

6. 메모리: 전체 RAM과 앱 한도 구분

프로세스의 가상 주소 공간에는 코드·정적 데이터·힙·스택 등이 있습니다. 동적 데이터 증가가 힙과 상주 메모리 증가로 나타날 수 있지만 앱의 Heap 값과 OS의 RSS는 같은 지표가 아닙니다. 주소 공간 전체 크기도 RSS와 다릅니다.

구분	의미	이번 판단에서의 역할
Windows 전체 메모리 비율	다른 앱과 WSL 등의 영향 포함	대상 하나의 원인을 확정할 수 없음
Linux 전체 RAM	WSL 안에서 보이는 물리 메모리	ps의 메모리 비율 분모
MEMORY_LIMIT	앱에 전달한 보호 설정	MemoryGuard 종료 로그와 비교

구분	의미	이번 판단에서의 역할
Current Heap	앱 내부 기록	앱이 보고한 증가와 초과 확인
RSS	프로세스 상주 메모리	OS 관측 증가 확인

예전 실제 출력 - 입력하지 않음

> [MemoryWorker] Current Heap: 150MB
> [MemoryGuard] Memory limit exceeded (150MB >= 128MB)
> [MemoryGuard] Self-terminating process 962 to prevent system instability.

이 출력은 원본 로그 (출력 해설)의 핵심 문구를 접두사 일부를 생략해 발췌한 것입니다. 관제 CSV (출력 해설)의 RSS는 18,356 → 146,376KiB로 증가했습니다. MiB = KiB / 1024입니다.

판단: 앱의 한도 초과와 MemoryGuard 보호 종료를 확인했습니다. 커널 OOM Killer의 작동은 커널 로그 등 추가 증거 없이 확정하지 않습니다. 메모리 누수·데이터 보관 증가가 의심되지만 비공개 코드의 정확한 해제 누락 위치는 확인하지 않았습니다.

메모리 부족은 일반적으로 페이지 회수·스왑·다른 프로세스의 자원 경쟁을 늘릴 수 있습니다. 이번 실험에서 시스템 전체 RAM 소진이나 그 지연량을 측정한 것은 아닙니다.

이 단계의 용어 설명

용어	이 단계에서의 뜻과 읽는 방법
RAM·힙·RSS	전체 물리 메모리, 프로세스의 동적 할당 영역, 실제 상주량입니다. 서로 다른 범위의 지표입니다.
MemoryGuard	앱 로그에 나타난 메모리 보호 정책입니다. Windows RAM 사용률과 같은 기준으로 보지 않습니다.
OOM Killer	메모리 부족 상황에서 커널이 프로세스를 종료하는 기능입니다. 이번 로그만으로 커널의 개입을 입증하지 않습니다.

7. 메모리 설정 비교와 임시 조치

비교 항목	Before	After
메모리 / CPU / 동시 설정	128 / 100 / false	256 / 100 / false
작업 PID	962	1156
로그 기준 생존 시간	17.202초	32.363초
종료 시 앱 내부 값	150MB	275MB
관측 최대 RSS	146,376KiB	274,516KiB
결과	MemoryGuard 종료	MemoryGuard 종료

생존 시간은 Agent listening부터 MemoryGuard 종료 요청 로그까지입니다. 증가분은 32.363 - 17.202 = 15.161초입니다. OS가 완전히 종료한 정확한 순간까지 측정한 값은 아닙니다. OOM 보고서

한도를 높이는 것은 그릇을 크게 만드는 것과 비슷합니다. 계속 들어오는 데이터를 줄이거나 회수하지 않으면 다시 한도에 도달합니다. **임시 조치**는 우리가 한 한도 상향이고, **보호 정책**은 앱의 MemoryGuard 동작입니다. 두 용어를 구분합니다.

25MB 단위로 증가해 128에서는 125→150, 256에서는 250→275 단계에서 초과했습니다. 1초 관제는 마지막 할당과 종료 사이를 놓칠 수 있으므로 관측 최대와 실제 절대 최대를 구분합니다.

근본 조치의 제안은 보관량 제한·캐시 만료·참조 회수 등의 소스 검토입니다. 실제로 수정·검증한 코드 변경이라고 말하지 않습니다.

이 단계의 용어 설명

용어	이 단계에서의 뜻과 읽는 방법
임계치	보호 동작의 기준값입니다. 앱이 25MB씩 증가하면 128을 정확히 찍지 않고 150에서 초과할 수 있습니다.
Before·After	설정 변경 전후의 별도 실행입니다. 한 변수만 바꿔 비교의 근거를 확보합니다.
임시 조치	문제의 영향을 줄이거나 발생을 늦추는 조치입니다. 생존 시간 증가만으로 누수 해결을 뜻하지 않습니다.

8. CPU: 세 종류의 숫자 구분

지표	측정 범위	설명 시 주의
앱 Current Load	내부 계산 값	OS 사용률과 같다고 가정하지 않음
ps %CPU	실행 이후 평균	짧은 버스트가 작게 보일 수 있음
top %CPU	갱신 구간	관측 간격·CPU 표시 모드 확인

예전 첫 관측은 1초 간격 30회로 최대 4%였고 기록이 실제 종료 전에 끝났습니다. 더 짧은 구간과 종료 로그를 보기 위해 PTY 기록 및 0.1초 관제를 추가했습니다. 관측 부족을 숨기지 않고 보완한 것입니다.

실행 구문 해설 - 경로와 관제 대상이 준비된 단계에서 사용

```
script -q -e -f -c './agent-leak-app' "$RUN_DIR/app.log" ₩
  < /dev/null > "$RUN_DIR/console.log" 2>&1 &
SESSION_PID=$!
LC_ALL=C top -b -d 0.1 -n 450 -w 180 -p "$TOP_PIDS" ₩
  > "$RUN_DIR/top.log" 2> "$RUN_DIR/top-error.log" &
```

TOP_PIDS는 이번 앱 PID들을 쉼표로 연결한 목록입니다. script는 가상 터미널 출력을 기록하며 -q는 안내 축소, -e는 자식 종료 상태 반영, -f는 자주 기록, -c는 명령 지정입니다. < /dev/null은 사용자 터미널 입력을 읽지 않도록 연결합니다. script 매뉴얼

Python stdout은 터미널과 일반 파일에서 버퍼링 방식이 다를 수 있습니다. PTY로 재수집했을 때 Watchdog 문구를 확보했습니다. 버퍼링을 유일한 원인으로 확정하거나 이 바이너리의 정확한 Python 버전을 단정하지 않습니다. Python 출력 문서

top -b는 배치, -d 0.1은 간격, -n 450은 최대 화면 수, -w 180은 폭, -p는 대상 선택입니다. 관제 자체 종료와 앱 종료는 별개입니다. 첫 프레임은 이후 갱신 구간과 구분해 집계에서 제외했습니다. 기록 시작·끝도 확인하며 수집 범위 밖은 추정값으로 채우지 않습니다.

기본 Irix 모드의 프로세스 100%는 논리 CPU 하나 기준이며, 표시 모드에 따라 해석이 달라집니다. 멀티스레드 프로세스는 100%를 넘을 수도 있습니다. top 매뉴얼

이 단계의 용어 설명

용어	이 단계에서의 뜻과 읽는 방법
누적 평균·구간 사용률	ps는 실행 이후 평균, top은 갱신 구간의 사용률을 보여줍니다. 시간 범위가 달라 값도 다릅니다.
Current Load	앱이 기록한 내부 부하 수치입니다. 계산식을 확인하지 않았으므로 OS 측정값과 동일시하지 않습니다.
CPU 코어	연산을 실행하는 단위입니다. 일반적인 top 모드의 프로세스 100%는 논리 CPU 하나 기준입니다.

9. CPU 종료와 설정 변경 검증

예전 실제 출력 - 원본에서 핵심 구간 발췌, 입력하지 않음

```
2026-10-09 11:58:20,549 [INFO] [CpuWorker] Current Load: 53.11%
2026-10-09 11:58:20,649 [CRITICAL] [CpuWorker] CPU Threshold Violated! (53.11%).
>>> [SYSTEM] WATCHDOG: INITIATING EMERGENCY ABORT (SIGTERM) <<<
```

비교 항목	Before	After
메모리 / CPU / 동시 설정	512 / 80 / false	512 / 40 / false
작업 PID	1921	2359
top 관측 최대	50.0%	40.0%
앱 동작	Watchdog 보호 종료	40%에서 냉각, 5%에서 재증가
생존·종료	36.380초 후 보호 종료	55초 시점 진행, 이후 수동 종료
최종 코드	143	143

설정 80인데 내부 53.11%에서 종료했으므로 환경변수를 "OS CPU 80% 초과 시 종료 기준"이라고 설명하지 않습니다. 구간 최대 50%가 지속 사용률이나 컴퓨터 전체 사용률이라는 뜻도 아닙니다. CPU 보고서와 증거 링크

CPU 경쟁은 다른 실행 가능 작업의 대기를 늘릴 수 있습니다. 이번에는 요청 응답 시간·P99·시스템 전체 포화를 별도로 측정하지 않았으므로 지연 수치를 만들어 말하지 않습니다. 설정 40의 결과는 이번 관측 범위의 보호 종료 회피이며 영구 무장애 보장이 아닙니다.

이 단계의 용어 설명

용어	이 단계에서의 뜻과 읽는 방법
Watchdog	과점유 등을 감시해 보호 동작을 수행하는 장치입니다. 이번에는 명시적 SIGTERM 로그가 근거입니다.
SIGTERM·143	종료 요청 시그널 15와 Bash의 128+15 상태입니다. 숫자만으로 요청 주체를 알 수 없습니다.
냉각·관측 기간	부하를 낮추는 동작과 실제 확인한 시간 범위입니다. 제한된 기간 생존을 영구 안정성으로 확대하지 않습니다.

10. Deadlock: 살아 있음과 진행함 구분

프로세스는 실행 중인 프로그램이고 스레드는 그 안의 실행 흐름입니다. 같은 프로세스의 스레드는 주소 공간을 공유합니다. 공유 자원을 보호하는 락의 획득 순서가 충돌하면 교착이 발생할 수 있습니다.

관측 명령 해설 - 현재 작업 자식 PID를 확인한 뒤 사용

```
ps -ef | grep '[a]gent-leak-app'
ps -L -p "$WORKER_PID" -o pid,tid,pcpu,rss,time,stat,wchan:32,comm
stat -c 'log_bytes=%s modified=%y' "$RUN_DIR/app.log"
tail -n 20 "$RUN_DIR/app.log"
```

조회	목적
ps -ef	PID 존재와 부모 관계 확인
-L, tid	스레드 단위 관측
time	누적 CPU 시간, 표시 정밀도 확인
wchan	커널 대기 위치 참고
stat	로그 크기·마지막 수정 시각
tail	마지막 작업 지점

두 번 이상 시각을 두고 동일 항목을 저장합니다. 첫 관측이 락 획득 중이고 다음 관측에 BLOCKED가 추가됐다면 그 구간에는 진행이 있었습니다. **BLOCKED 이후 추가 관측**으로 정체를 확인해야 합니다. 단순히 sleep 10을 입력했다는 이유만으로 실제 스냅샷 간격이 10초라고 쓰지 않고 수집 시각을 확인합니다.

예전 true 실행에서는 PID 9226이 유지되고 RSS 18,512KiB, 로그 2,792바이트와 수정 시각이 두 관측에서 동일했습니다. 마지막 로그에서 T1은 A를 보유하며 B를 기다리고 T2는 B를 보유하며 A를 기다렸습니다. 원본 스냅샷 (출력 해설)

교착 조건	이번 사례와의 연결
상호 배제	각 자원 락을 한 워커가 보유
점유 대기	보유한 상태에서 다른 락 요청
비선점	대기자가 상대 락을 임의 회수하지 못하는 일반 락 모델로 추론
순환 대기	T1→B→T2→A→T1

판단: 락 보유·추가 요청·순환 관계와 진행 정체를 함께 근거로 진단합니다. futex는 동기화 대기에 쓰이므로 futex_do_wait 만으로 교착을 확정하지 않습니다. 락 구현 종류·타임아웃과 Worker 이름별 TID 대응은 미확인입니다. futex 매뉴얼

이 단계의 용어 설명

용어	이 단계에서의 뜻과 읽는 방법
락·상호 배제	공유 자원을 동시에 사용하지 않도록 접근을 제한하는 장치와 조건입니다.
점유 대기·순환 대기	하나를 가진 채 다른 것을 기다리거나 그 관계가 고리를 이루는 상황입니다.
futex·WCHAN	Linux 동기화 대기 기능과 커널 대기 위치입니다. futex 한 줄만으로 교착을 확정하지 않습니다.
TID·mtime	스레드 번호와 파일 수정 시각입니다. 같은 PID의 RSS를 TID 행 수만큼 합산하지 않습니다.

11. Deadlock 회피, 수동 종료와 종료 코드

메모리 512와 CPU 40을 고정하고 동시 설정만 true→false로 바꿨습니다.

관측	true	false
PID	유지	유지
RSS	18,512→18,512KiB	121,000→197,812KiB
로그 크기	2,792→2,792바이트	3,474→3,870바이트
작업	순환 대기 뒤 정체	워커 로그 진행
마지막 종료	수동 SIGTERM	수동 SIGTERM

false에서도 스레드 3개와 메인 futex 대기가 관측됐습니다. 앱 전체의 단일 스레드 전환이라고 말하지 않습니다. 로그 진행을 근거로 이번 교착 경로 회피를 확인했습니다. RSS 증가만으로 정상이라고 판단하거나 그 증가만으로 정상 시나리오 전체를 누수라고 확정하지 않습니다. Deadlock 보고서

실험 정리 시 기록한 PID와 현재 이름·부모 관계를 확인한 뒤 SIGTERM을 요청하고 시각·대상을 cleanup.log에 남깁니다. 예전 PID를 그대로 종료하지 않습니다. 부모 종료 후 자식도 사라졌어도 내부 신호 전달 방식까지 확인한 것은 아닙니다.

종료 상태 저장 구문 해설

```
wait "$SESSION_PID"
SESSION_EXIT=$?
printf '%s₩n' "$SESSION_EXIT" > "$RUN_DIR/exit-code.txt"
```

wait는 현재 셸이 시작한 자식 작업의 종료 상태를 회수합니다. $?는 직전 명령 상태이므로 바로 저장합니다. 일반 실행에서는 앱 시작 PID, PTY에서는 기록 작업 PID를 사용합니다. 새 셸에서 과거 PID의 종료 코드를 임의로 복원하지 않습니다.

137은 128+9, 143은 128+15인 시그널 종료 규칙과 일치하지만, 프로그램이 같은 숫자를 직접 반환할 수도 있어 숫자만으로 원인·발신자를 확정하지 않습니다. Watchdog 로그와 수동 기록을 대조합니다. 규칙은 script의 종료 상태 설명에 명시돼 있습니다.

이 단계의 용어 설명

용어	이 단계에서의 뜻과 읽는 방법
회피	문제가 발생하는 실행 경로를 사용하지 않는 조치입니다. false라도 앱 전체가 단일 스레드라는 뜻은 아닙니다.
수동 정리·cleanup.log	실험자가 종료 요청을 보냈음을 남긴 기록입니다. 자동 보호 종료와 구분합니다.
진행성	작업이 앞으로 나아가는 성질입니다. PID 존재 외에 시간 간격을 둔 로그 변화로 확인합니다.

12. 보고서를 증거와 함께 설명하기

각 장애 보고서는 현상·증거·원인·조치 및 검증의 4개 절을 갖춥니다. 설명은 다음 순서로 구성합니다.

1. **결론:** 무엇이 발생했는가?
2. **증거:** 어떤 파일의 어떤 시각·PID·값이 뒷받침하는가?
3. **비교:** 무엇 하나를 변경했고 무엇이 달라졌는가?
4. **한계:** 어떤 내용은 확인하지 못했는가?

분류	말할 수 있는 내용
확인한 사실	로그 문구, 설정, PID, RSS, 구간 CPU, 종료 코드

분류	말할 수 있는 내용
증거 기반 판단	MemoryGuard 보호 종료, Watchdog 보호 종료, 순환 대기 진단
미확인	특정 소스 위치, 실제 요청 지연량, 내부 보호 기준 구현

본 실험은 바이너리를 실행하고 외부 상태를 관측했으며 디컴파일·리버스 엔지니어링을 하지 않았습니다. 선택 스케줄링 분석도 응용 작업 로그의 FCFS형 패턴에 대한 추론입니다. Linux 커널 알고리즘의 확정이 아닙니다.

다음은 평가 질문과 답변 근거를 보고, 빈 실습 기록 양식을 본인 표현으로 작성합니다. 문서를 게시한 것과 학습자가 직접 수행·이해한 것은 구분합니다.

이 단계의 용어 설명

용어	이 단계에서의 뜻과 읽는 방법
증거·추론·한계	관측 파일, 그로부터 판단한 내용, 아직 확인하지 못한 범위를 구분해 설명합니다.
근본 원인·우회 조치	문제를 일으키는 원리와 그 영향을 줄이는 방법입니다. 재현만으로 내부 코드 위치까지 알 수는 없습니다.
Issue·무결성	협업용 문제 보고와 원본이 바뀌지 않았다는 성질입니다. 링크와 해시로 추적 가능하게 합니다.

제2부. 준비부터 여섯 번의 비교 실험까지

실습 절차 1. 자료와 관제 파일 준비

앞의 환경·파일·환경변수 장을 먼저 완료합니다. 실습 경로는 ~/agent-b42-study입니다. 이후 실습은 그 준비를 반복하지 않고 이어서 실행합니다. 아래 수집 절차는 새 재실습을 위한 계획이며, 과거 증거 파일을 재현한 것처럼 수정하지 않습니다.

저장소의 관제 파일을 받으려면 Ubuntu에서 먼저 command -v git으로 설치 여부를 확인합니다. 경로가 나오면 아래를 실행합니다. 명령을 찾지 못하면 도구 설치부터 확인하고 다음으로 진행합니다. 이미 같은 자료 폴더가 있으면 clone을 반복하지 않고 해당 자료를 확인합니다.

입력할 명령 - Ubuntu, 자료 폴더를 처음 받을 때

```
git clone https://github.com/bidulgiya999/codyssey_work_B4-2.git ₩
  "$HOME/codyssey-b42-material"
REPO_DIR="$HOME/codyssey-b42-material"
source ~/agent-b42-study/env.sh
cp "$REPO_DIR/scripts/monitor.sh" "$AGENT_HOME/monitor.sh"
chmod u+x "$AGENT_HOME/monitor.sh"
bash -n "$AGENT_HOME/monitor.sh"
cd "$AGENT_HOME"
```

git clone은 학습 자료를 받는 추가 절차입니다. 제공 바이너리는 저장소에 없으므로 앞 단계에서 다운로드 폴더의 파일을 복사해야 합니다. bash -n에서 출력이 없고 오류 없이 돌아오면 문법 검사를 통과한 것입니다. 실제 수집 성공은 실행 후 확인합니다.

순서	실행 이유	빼먹으면 생기는 문제
자료 받기	사용한 소스와 보고서를 같은 기준으로 읽기	monitor.sh 경로가 없을 수 있음
source	현재 셸에 새 학습 경로 적용	예전 경로나 빈 변수를 사용할 수 있음
cp·chmod	관제 파일과 실행 권한 준비	앱은 시작했는데 관제를 못 할 수 있음
문법 검사	실행 전에 셸 문법 오류 발견	짧은 장애의 관측 기회를 놓칠 수 있음
cd	./agent-leak-app의 기준 위치 결정	다른 폴더에서는 실행 파일을 못 찾음

이 단계의 용어 설명

용어	이 단계에서의 뜻과 읽는 방법
ELF	Linux 실행 파일 형식입니다. file 출력의 x86-64 또는 aarch64와 uname -m을 대조합니다.
경로·홈	파일 위치를 나타냅니다. ~는 현재 사용자의 홈이며 /mnt/c는 Windows C드라이브 접근 경로입니다.
권한 r/w/x	읽기·쓰기·실행 권한입니다. chmod u+x는 소유자에게 실행 권한을 추가합니다.
source	파일의 명령을 현재 셸에서 읽습니다. 새 터미널에서는 다시 적용해야 합니다.

실습 절차 2. 공통 수집 함수를 이해하고 정의

아래는 설명을 반복하지 않기 위해 명령을 함수로 묶은 **교안용 수집 코드**입니다. 제공 바이너리의 수정 코드가 아닙니다. 함수 정의만으로 앱이 실행되지는 않습니다. 이 블록을 현재 Ubuntu 셸에 한 번 입력한 뒤, 각 실험에서 b42_start·b42_snapshot·b42_finish를 호출합니다.

이 수집 코드는 CPU 관제를 앱 종료 확인까지 계속 돌리도록 구성했습니다. 원본 실험의 -n 450과 수집 기간이 다를 수 있으므로 새 표본 수·시작·끝을 기록합니다. 이 교안을 만들며 바이너리를 새로 실행한 검증 결과를 제시하는 것은 아닙니다.

입력할 명령 - 함수 정의

```
B42_ACTIVE=${B42_ACTIVE:-0}

b42_start() {
  local label="$1"
  if [[ "$B42_ACTIVE" == 1 ]]; then
    echo "이전 실험을 먼저 기록하고 종료해주세요."
    return 1
  fi
  if [[ ! "$label" =~ ^[a-z0-9-]+$ || -z "$AGENT_HOME" ]]; then
    echo "실험 이름과 환경변수를 확인해주세요."
    return 1
  fi
  if [[ "$(id -u)" == 0 ]]; then
    echo "앱은 root가 아닌 일반 계정으로 실행합니다."
    return 1
  fi
  if [[ ! -x ./agent-leak-app || ! -x ./monitor.sh ]]; then
    echo "현재 폴더의 앱과 관제 실행 권한을 확인해주세요."
    return 1
  fi
  if [[ "$(pwd -P)" != "$(realpath "$AGENT_HOME")" ]]; then
    echo "AGENT_HOME 폴더로 이동한 뒤 실행해주세요."
    return 1
  fi
  if pgrep -x agent-leak-app >/dev/null; then
    echo "기존 앱이 실행 중입니다. 먼저 해당 실행을 확인해주세요."
    return 1
  fi
  if ss -ltnH 'sport = :15034' | grep -q .; then
    echo "15034 포트가 사용 중입니다. 점유 프로세스를 확인해주세요."
    return 1
  fi

  mkdir -p "$AGENT_HOME/evidence"
  RUN_DIR=$(mktemp -d ₩
    "$AGENT_HOME/evidence/$label-$(date +%Y%m%d-%H%M%S)-XXXXXX")
  [[ -d "$RUN_DIR" ]] || return 1
  printf 'MEMORY_LIMIT=%s₩nCPU_MAX_OCCUPY=%s₩nMULTI_THREAD_ENABLE=%s₩n' ₩
    "$MEMORY_LIMIT" "$CPU_MAX_OCCUPY" "$MULTI_THREAD_ENABLE" ₩
    > "$RUN_DIR/settings.txt"
  date -Iseconds > "$RUN_DIR/started_at.txt"

  script -q -e -f -c './agent-leak-app' "$RUN_DIR/app.log" ₩
    < /dev/null > "$RUN_DIR/console.log" 2>&1 &
  SESSION_PID=$!
  printf '%s₩n' "$SESSION_PID" > "$RUN_DIR/session.pid"
  APP_PARENT_PID=""
  WORKER_PID=""
  MONITOR_PIDS=()
```

```
  TOP_MONITOR_PID=""

  for ATTEMPT in {1..30}; do
    APP_PARENT_PID=$(pgrep -P "$SESSION_PID" ₩
      -x agent-leak-app | head -n 1)
    if [[ -n "$APP_PARENT_PID" ]]; then
      WORKER_PID=$(pgrep -P "$APP_PARENT_PID" ₩
        -x agent-leak-app | head -n 1)
    fi
    [[ -n "$WORKER_PID" ]] && break
    sleep 0.1
  done

  if [[ -z "$APP_PARENT_PID" || -z "$WORKER_PID" ]]; then
    echo "예상한 부모·자식 구조를 찾지 못했습니다. 로그를 확인합니다."
    tail -n 40 "$RUN_DIR/app.log"
    printf '%s manual SIGTERM SESSION_PID=%s₩n' ₩
      "$(date -Iseconds)" "$SESSION_PID" >> "$RUN_DIR/cleanup.log"
    kill -TERM "$SESSION_PID" 2>/dev/null
    if wait "$SESSION_PID"; then APP_EXIT=0; else APP_EXIT=$?; fi
    printf '%s₩n' "$APP_EXIT" > "$RUN_DIR/exit-code.txt"
    echo "ps와 포트 상태를 확인한 뒤 다음 실험으로 진행해주세요."
    return 1
  fi

  printf '%s₩n' "$APP_PARENT_PID" "$WORKER_PID" ₩
    > "$RUN_DIR/observed-pids.txt"
  printf '%s₩n' "$WORKER_PID" > "$RUN_DIR/worker.pid"
  for PID in "$APP_PARENT_PID" "$WORKER_PID"; do
    ./monitor.sh "$PID" ₩
      > "$RUN_DIR/monitor-$PID.csv" ₩
      2> "$RUN_DIR/monitor-$PID.log" &
    MONITOR_PIDS+=("$!")
  done
  TOP_PIDS=$(paste -sd, "$RUN_DIR/observed-pids.txt")
  if [[ "$label" == cpu-* ]]; then
    LC_ALL=C top -b -d 0.1 -w 180 -p "$TOP_PIDS" ₩
      > "$RUN_DIR/top.log" 2> "$RUN_DIR/top-error.log" &
    TOP_MONITOR_PID=$!
  fi
  ps -p "$TOP_PIDS" ₩
    -o pid,ppid,etimes,pcpu,pmem,rss,nlwp,stat ₩
    > "$RUN_DIR/ps-start.txt"
  B42_ACTIVE=1
  printf '저장 폴더: %s₩n부모 PID: %s₩n작업 PID: %s₩n' ₩
    "$RUN_DIR" "$APP_PARENT_PID" "$WORKER_PID"
}

b42_snapshot() {
  local snapshot_name="$1"
  [[ "$B42_ACTIVE" == 1 ]] || return 1
  [[ "$snapshot_name" =~ ^[a-z0-9-]+$ ]] || return 1
  {
    date -Iseconds
    ps -p "$TOP_PIDS" ₩
      -o pid,ppid,etimes,pcpu,pmem,rss,nlwp,stat
    ps -L -p "$WORKER_PID" ₩
      -o pid,tid,pcpu,rss,time,stat,wchan:32,comm
    stat -c 'log_bytes=%s modified=%y' "$RUN_DIR/app.log"
    tail -n 25 "$RUN_DIR/app.log"
  } | tee "$RUN_DIR/$snapshot_name.txt"
```

```
}

b42_finish() {
  local mode="$1"
  [[ "$B42_ACTIVE" == 1 ]] || return 1
  [[ "$mode" == auto || "$mode" == manual ]] || return 1
  if [[ "$mode" == auto ]] && kill -0 "$SESSION_PID" 2>/dev/null; then
    echo "아직 실행 중입니다. 종료 로그를 확인하거나 수동 정리를 선택합니다."
    return 1
  fi
  if [[ "$mode" == manual ]]; then
    for PID in "$WORKER_PID" "$APP_PARENT_PID"; do
      if [[ "$(ps -p "$PID" -o comm= 2>/dev/null)" == agent-leak-app ]]; then
        printf '%s manual SIGTERM PID=%s₩n' "$(date -Iseconds)" "$PID" ₩
          >> "$RUN_DIR/cleanup.log"
        kill -TERM "$PID" 2>/dev/null
      fi
    done
  fi
  if wait "$SESSION_PID"; then APP_EXIT=0; else APP_EXIT=$?; fi
  printf '%s₩n' "$APP_EXIT" > "$RUN_DIR/exit-code.txt"
  for MONITOR_PID in "${MONITOR_PIDS[@]}"; do
    wait "$MONITOR_PID"
  done
  if [[ -n "$TOP_MONITOR_PID" ]]; then
    if [[ "$(ps -p "$TOP_MONITOR_PID" -o comm= 2>/dev/null)" == top ]]; then
      kill -TERM "$TOP_MONITOR_PID" 2>/dev/null
    fi
    wait "$TOP_MONITOR_PID"
  fi
  ps -p "$TOP_PIDS" ₩
    -o pid,ppid,etimes,pcpu,pmem,rss,nlwp,stat ₩
    | tee "$RUN_DIR/ps-after.txt"
  B42_ACTIVE=0
  printf '앱 기록 작업 종료 코드: %s₩n증거 폴더: %s₩n' ₩
    "$APP_EXIT" "$RUN_DIR"
}

b42_cpu_summary() {
  awk '
  /^top -/ {frame++; next}
  frame > 1 && $1 ~ /^[0-9]+$/ && $NF == "agent-leak-app" {
      p=$1; c=$9+0
      if (!(p in peak) || c > peak[p]) peak[p]=c
      n[p]++
  }
  END {
      for (p in peak)
          printf "PID=%s MAX_CPU=%.1f%% SAMPLES=%d₩n", p, peak[p], n[p]
  }' "$1/top.log" | tee "$1/top-summary.txt"
}
```

수집 코드의 실행 순서와 출력 해설

구간	왜 필요한가	출력·동작 해설
시작 전 검사	중복 앱·포트 충돌·실행 권한 오류 방지	실패하면 앱을 새로 시작하지 않고 안내
mktemp -d	같은 시각에도 증거 폴더 충돌 방지	실제 생성된 경로가 RUN_DIR에 저장
설정 저장 후 실행	실행 당시 조건을 보존	settings.txt와 앱 부팅 설정을 대조
script 직후 $! 저장	다른 백그라운드 작업이 PID 값을 바꾸기 전에 저장	SESSION_PID는 앱 PID가 아님
부모·자식 재시도	자식 생성 전에 조회할 가능성 고려	구조가 다르면 로그를 보고 중단
PID별 관제	실제 작업 자원의 증가를 놓치지 않기 위해	부모·자식 CSV를 따로 생성
CPU 실험의 top	평균에 가려지는 짧은 부하 확인	종료 때까지 수집, 표본 수를 실제로 기록
스냅샷 함수	같은 항목으로 진행 여부 비교	ps·stat·tail은 순차 조회이므로 원자적 동일 순간은 아님
auto 종료 확인	아직 살아 있는 앱에 무기한 wait하지 않기 위해	살아 있으면 return 1, 추가 확인 필요
manual 종료 기록	실험자 정리와 보호 종료를 구분	cleanup.log의 시각·PID를 근거로 설명
wait 직후 상태 저장	다른 명령이 $?를 바꾸기 전에 확보	top의 종료 코드와 앱 기록 작업의 코드를 구분
관제 종료	학습 후 도구가 계속 남지 않도록 정리	앱과 top·monitor의 Done을 구분

수집 함수는 관측을 돕는 도구입니다. b42_start에서 실패 메시지가 나오면 다음 실험 코드를 연달아 실행하지 않습니다. 먼저 앱 로그·ps·포트를 확인합니다. 새 셸을 열면 함수와 PID 변수는 사라지므로 실행 중인 앱을 새로 찾고 현재 상태부터 확인해야 합니다.

CPU 집계는 이번 기본 top 출력의 1번째 열 PID, 9번째 열 %CPU, 마지막 열 명령 이름을 사용합니다. 개인 설정으로 열 순서가 다르면 해당 부분을 수정해야 합니다.

이 단계의 용어 설명

용어	이 단계에서의 뜻과 읽는 방법
PID·PPID	프로세스 번호와 부모 번호입니다. 실행기와 실제 작업 자식을 구분해 관제합니다.
PTY·script	가상 터미널과 그 출력을 기록하는 도구입니다. 세션 PID와 앱 PID는 다를 수 있습니다.
수동 정리·cleanup.log	실험자가 종료 요청을 보냈음을 남긴 기록입니다. 자동 보호 종료와 구분합니다.

실습 절차 3. 메모리 두 번 실행하고 비교

입력할 명령 - 첫 실행

```
export MEMORY_LIMIT=128
export CPU_MAX_OCCUPY=100
export MULTI_THREAD_ENABLE=false
b42_start oom-128
```

저장 폴더와 PID가 출력됐는지 확인한 뒤 진행합니다. 부팅 검사·첫 메모리 증가·관제 수집을 기다립니다.

```
sleep 25
b42_snapshot observed
tail -n 25 "$RUN_DIR/app.log"
b42_finish auto
OOM_BEFORE_DIR="$RUN_DIR"
```

25초는 과거 종료 시간보다 길게 잡은 관측 대기이지 새 실행의 종료 보장이 아닙니다. auto에서 아직 실행 중이라고 나오면 로그를 더 확인합니다. 재현되지 않았다면 그 사실과 수동 정리를 기록하며 성공으로 쓰지 않습니다. 실험을 끝내기로 했다면 b42_finish manual로 정리합니다.

입력할 명령 - 이전 실행을 마친 뒤 두 번째 실행

```
export MEMORY_LIMIT=256
b42_start oom-256
```

시작 성공을 확인한 뒤 다음을 실행합니다.

```
sleep 40
b42_snapshot observed
tail -n 25 "$RUN_DIR/app.log"
b42_finish auto
OOM_AFTER_DIR="$RUN_DIR"
```

각 폴더의 worker.pid를 읽어 해당 CSV를 확인합니다. 다음은 변경 후 자료 조회입니다.

```
READ_WORKER=$(cat "$OOM_AFTER_DIR/worker.pid")
cat "$OOM_AFTER_DIR/monitor-$READ_WORKER.csv"
cat "$OOM_BEFORE_DIR/settings.txt"
cat "$OOM_AFTER_DIR/settings.txt"
```

설명 순서: 부팅 성공 → 앱 내부 메모리 증가 → RSS 증가 → 한도 초과·자체 종료 로그 → PID 소멸 → 한도 변경 후 생존 시간 비교.

생존 시간은 양쪽 모두 동일한 시작·끝 로그를 사용해 계산합니다. 폴더 이름 시각이나 관제 마지막 표본을 앱 종료 요청 시각으로 대신 사용하지 않습니다. 예전 실제값은 17.202초와 32.363초이고 새 값은 로그에서 직접 계산합니다.

예상 질문: "RAM 사용률이 낮은데 왜 종료했나요?" 답변은 전체 RAM 소진과 앱 보호 한도를 구분해 설명합니다.

이 단계의 용어 설명

용어	이 단계에서의 뜻과 읽는 방법
임계치	보호 동작의 기준값입니다. 앱이 25MB씩 증가하면 128을 정확히 찍지 않고 150에서 초과할 수 있습니다.
Before·After	설정 변경 전후의 별도 실행입니다. 한 변수만 바꿔 비교의 근거를 확보합니다.
임시 조치	문제의 영향을 줄이거나 발생을 늦추는 조치입니다. 생존 시간 증가만으로 누수 해결을 뜻하지 않습니다.

실습 절차 4. CPU 두 번 실행하고 비교

입력할 명령 - 설정 80

```
export MEMORY_LIMIT=512
export CPU_MAX_OCCUPY=80
export MULTI_THREAD_ENABLE=false
b42_start cpu-80
```

시작 성공을 확인하고 다음을 실행합니다.

```
sleep 75
b42_snapshot observed
tail -n 40 "$RUN_DIR/app.log"
b42_finish auto
CPU_BEFORE_DIR="$RUN_DIR"
b42_cpu_summary "$CPU_BEFORE_DIR"
```

75초가 종료 보장은 아닙니다. 살아 있으면 종료 로그를 더 확인하거나 미재현으로 기록하고 manual로 정리합니다. 수동 종료를 Watchdog 종료라고 쓰지 않습니다.

입력할 명령 - 앞 실행을 마친 뒤 설정 40

```
export CPU_MAX_OCCUPY=40
b42_start cpu-40
```

시작 성공을 확인하고 다음을 실행합니다.

```
sleep 55
b42_snapshot observed
CPU_AFTER_DIR="$RUN_DIR"
b42_finish manual
b42_cpu_summary "$CPU_AFTER_DIR"
cat "$CPU_AFTER_DIR/cleanup.log"
```

관측 해설: Before에서 WATCHDOG·SIGTERM 로그를 찾습니다. After에서는 40% 도달·냉각·재증가 로그와 55초 시점 PID·작업 진행을 봅니다. 마지막 143이 수동 정리 때문인지 cleanup.log로 확인합니다. 원본 비교 수치 50→40%를 새 관측 결과라고 미리 채우지 않습니다.

top 기록 범위와 오류도 확인합니다.

```
awk '/^top -/ {if (!n) first=$0; last=$0; n++}
END {print first; print last; print "화면 수:", n}' ₩
  "$CPU_AFTER_DIR/top.log"
cat "$CPU_AFTER_DIR/top-error.log"
```

최초 화면을 제외하는 이유, ps 평균과 top 구간 값의 차이, 논리 CPU 기준과 전체 CPU 요약의 차이를 함께 설명합니다.

이 단계의 용어 설명

용어	이 단계에서의 뜻과 읽는 방법
누적 평균·구간 사용률	ps는 실행 이후 평균, top은 갱신 구간의 사용률을 보여줍니다. 시간 범위가 달라 값도 다릅니다.
Current Load	앱이 기록한 내부 부하 수치입니다. 계산식을 확인하지 않았으므로 OS 측정값과 동일시하지 않습니다.
CPU 코어	연산을 실행하는 단위입니다. 일반적인 top 모드의 프로세스 100%는 논리 CPU 하나 기준입니다.
Watchdog	과점유 등을 감시해 보호 동작을 수행하는 장치입니다. 이번에는 명시적 SIGTERM 로그가 근거입니다.

실습 절차 5. 교착 재현과 회피 비교

입력할 명령 - true 설정

```
export MEMORY_LIMIT=512
export CPU_MAX_OCCUPY=40
```

```
export MULTI_THREAD_ENABLE=true
b42_start deadlock-true
```

시작 성공을 확인한 뒤 상태를 비교합니다.

```
sleep 12
b42_snapshot snapshot-1
sleep 10
b42_snapshot snapshot-2
DEADLOCK_BEFORE_DIR="$RUN_DIR"
```

12초도 교착 진입 보장은 아닙니다. 첫 관측 뒤 BLOCKED가 새로 생겼다면 추가 관측이 필요합니다. 다음은 그때만 실행할 추가 관측입니다.

```
sleep 10
b42_snapshot snapshot-3
```

설명 순서: PID 유지 → 스레드·RSS·로그 정체 → 마지막 락 보유와 추가 요청 → 순환 관계 → 일반 대기와 구분. 이후 아래로 정리합니다.

```
b42_finish manual
cat "$DEADLOCK_BEFORE_DIR/cleanup.log"
```

입력할 명령 - false 설정

```
export MULTI_THREAD_ENABLE=false
b42_start deadlock-false
```

시작 성공을 확인한 뒤 비교·정리합니다.

```
sleep 12
b42_snapshot snapshot-1
sleep 10
b42_snapshot snapshot-2
DEADLOCK_AFTER_DIR="$RUN_DIR"
b42_finish manual
```

새 스냅샷에서 로그 크기·수정 시각·작업 내용이 바뀌는지 봅니다. false여도 스레드가 여러 개일 수 있고 정상 메인 스레드도 기다릴 수 있습니다. 스레드 행의 RSS는 공유 프로세스 값이므로 합산하지 않습니다.

이 단계의 용어 설명

용어	이 단계에서의 뜻과 읽는 방법
락·상호 배제	공유 자원을 동시에 사용하지 않도록 접근을 제한하는 장치와 조건입니다.
점유 대기·순환 대기	하나를 가진 채 다른 것을 기다리거나 그 관계가 고리를 이루는 상황입니다.
futex·WCHAN	Linux 동기화 대기 기능과 커널 대기 위치입니다. futex 한 줄만으로 교착을 확정하지 않습니다.
회피	문제가 발생하는 실행 경로를 사용하지 않는 조치입니다. false라도 앱 전체가 단일 스레드라는 뜻은 아닙니다.

실습 절차 6. 보고·백업·설명으로 마무리

실행별로 settings.txt, app.log, monitor CSV, ps·스냅샷, exit-code.txt를 보존합니다. CPU에는 top.log와 관측 기간, 수동 정리에는 cleanup.log가 추가됩니다. 이 책의 실험 도우미는 새 증거를 만들며 기존 evidence를 바꾸지 않습니다.

Windows 백업을 원하면 먼저 새 백업 경로가 맞는지 확인한 후 다음을 사용합니다.

```
BACKUP_DIR="/mnt/c/Users/1234v/Downloads/agent-app-leak/study-results"
mkdir -p "$BACKUP_DIR/evidence"
cp -r "$AGENT_HOME/evidence/." "$BACKUP_DIR/evidence/"
cp "$AGENT_HOME/env.sh" "$AGENT_HOME/monitor.sh" "$BACKUP_DIR/"
diff -qr "$AGENT_HOME/evidence" "$BACKUP_DIR/evidence"
```

diff에 차이·오류 없이 돌아오면 비교한 내용이 동일합니다. 경로가 이미 사용 중이면 기존 파일을 섞지 않도록 새 백업 경로를 선택합니다.

보고서는 OOM·CPU·Deadlock 각각 현상, 증거, 원인, 조치 및 검증의 4개 절로 작성합니다. 누가·언제·어디서·어떤 조건으로 실행했고 무엇이 관측됐는지 기록합니다. 각 결론에 원본 파일을 연결합니다.

단계	평가 때 말할 내용
현상	종료인지, 살아 있지만 정체인지
증거	시각·PID·지표·로그 문구와 원본 위치
원인	직접 보호 종료 또는 순환 락 대기
조치	어떤 변수 하나를 어떻게 변경했는지
검증	생존 시간·냉각·진행 로그가 어떻게 바뀌었는지
한계	소스 구현·실제 지연·절대 최대 등 미확인 내용

제출할 때 교안만으로 보고서 3건을 대체하지 않습니다. 실제 보고서와 증거를 제출하고 교안은 본인 설명을 준비하는 데 사용합니다. 마지막으로 평가 질문에 자신의 표현으로 답하고, 모르는 항목은 실습 기록 양식에 남깁니다.

이 단계의 용어 설명

용어	이 단계에서의 뜻과 읽는 방법
증거·추론·한계	관측 파일, 그로부터 판단한 내용, 아직 확인하지 못한 범위를 구분해 설명합니다.
근본 원인·우회 조치	문제를 일으키는 원리와 그 영향을 줄이는 방법입니다. 재현만으로 내부 코드 위치까지 알 수는 없습니다.
Issue·무결성	협업용 문제 보고와 원본이 바뀌지 않았다는 성질입니다. 링크와 해시로 추적 가능하게 합니다.

제3부. 평가 질문과 증거를 연결하는 설명

1. 과제와 환경

Q1. 어떤 과제이며 무엇을 제출했나요?

답변의 핵심: 제공된 Linux 앱을 실행해 OOM·CPU·Deadlock을 관측하고, 환경변수 변경 전후를 검증한 장애 보고서 3건과 공통 Issue 템플릿을 작성했습니다. 원본 로그와 관제 자료를 연결했습니다.

증거: README의 요구사항 대응표, 공통 템플릿.

구분: SSH·방화벽·cron을 구축하는 별도 과제의 완료를 주장하지 않습니다. 보고서는 GitHub Issue 형식의 Markdown이며 실제 Issues 등록과는 다릅니다.

Q2. Windows에서 왜 Ubuntu를 사용했나요?

답변의 핵심: 제공 파일이 Linux 실행 형식이어서 WSL의 Ubuntu에서 실행했습니다. uname으로 환경을, file로 파일 형식을 확인했습니다.

증거: 환경 정보, 준비 명령.

Q3. 왜 whoami와 id -u를 모두 썼나요?

답변의 핵심: 사용자 이름과 숫자 UID를 확인했습니다. 원본은 jae0608, UID 1000이었고 root의 UID 0과 달라 일반 계정 실행 조건을 확인했습니다.

구분: UID가 1000이어야 하는 것이 아니라 root가 아닌 실행 계정인지 확인하는 것입니다.

Q4. x86 파일을 선택한 근거는 무엇인가요?

답변의 핵심: uname -m의 x86_64와 file의 x86-64 형식이 일치했습니다. 파일 이름만 보고 선택하지 않았고 ARM 파일은 사용하지 않았습니다. 코드 역분석은 하지 않았습니다.

증거: 파일 선택 기록.

Q5. chmod u+x와 chmod 600은 왜 다르게 사용했나요?

답변의 핵심: 실행 파일에는 소유자 실행 권한을 추가했고, 키 파일은 소유자만 읽고 쓸 수 있게 설정했습니다. 실행 권한 확보와 부팅 성공은 별도로 확인했습니다.

구분: 디렉터리 x는 경로 접근 의미이고 파일 x는 실행 권한입니다. 권한 표시만으로 모든 런타임 조건이 검증된 것은 아닙니다.

Q6. env.sh 저장, source, export는 어떻게 다른가요?

답변의 핵심: 저장은 설정 문서를 만드는 일이고 source는 그 내용을 현재 셸에서 실행하는 일입니다. export된 값은 이후 자식 프로그램에 전달됩니다. 셸에서 export로 바꿔도 파일 자체는 수정되지 않습니다.

증거: 초기 env.sh, 실행별 settings.txt. 초기 파일은 128/100/false여도 나중 실행은 다른 값일 수 있습니다.

Q7. 부팅 성공을 어떻게 확인했나요?

답변의 핵심: 일반 계정·환경변수·키·포트·로그 권한·미션 환경 6단계 검사와 Agent READY를 확인했습니다. 이어서 실제 작업 로그도 확인했습니다. 이 앱의 AGENT_KEY_PATH는 키 디렉터리이고 그 안에 secret.key가 있습니다.

증거: OOM 부팅 로그 (출력 해설).

구분: 포트 사전 조회의 제목 행만으로 이후 바인딩 성공까지 보장하지 않습니다. READY도 장기 정상 동작이나 요청 성공률을 보장하지 않습니다.

2. 프로세스와 증거 수집

Q8. 부모와 자식을 왜 각각 관제했나요?

답변의 핵심: 실제 사용량이 작업 자식에서 증가했기 때문입니다. PID와 PPID로 관계를 확인했고 부모만 관측해 자식의 메모리 증가를 놓치는 일을 피했습니다. PTY 실행의 SESSION_PID는 script 작업이므로 앱 PID와 구분했습니다.

증거: OOM 시작 ps (출력 해설), CPU 시작 ps (출력 해설).

Q9. monitor.sh를 직접 설명해보세요.

답변의 핵심: PID 인자 형식을 검사한 뒤 ps로 사용량과 상태를 반복 조회하고, 시각을 붙여 CSV로 출력했습니다. 좀비 행은 제외하며 유효 행이 없어지면 끝납니다. 별도 조회 실패 가능성도 있어 종료는 앱 로그·ps와 함께 확인했습니다.

증거: 관제 소스.

구분: bash -n은 문법 검사입니다. 실제로 실행해 대상 PID의 CSV가 기록되는지도 확인해야 합니다.

3. 메모리 장애

Q10. PC 전체 RAM 사용률이 높아서 종료됐나요?

답변의 핵심: 전체 사용률만으로 원인을 확정하지 않았습니다. 대상 프로세스 RSS 증가와 앱의 한도 초과·자체 종료 로그를 연결했습니다. MEMORY_LIMIT은 이 앱에 전달한 설정입니다.

증거: PID 962 CSV (출력 해설), MemoryGuard 로그 (출력 해설).

Q11. RSS와 Heap 값이 왜 다르며, 누수 위치도 찾았나요?

답변의 핵심: RSS는 OS의 상주 메모리 지표이고 Heap 로그는 앱 내부 값입니다. 단위·측정 시점도 다릅니다. 관측 기간의 누적 증가와 보호 종료는 확인했지만 특정 소스의 해제 누락 위치까지 확인하지 않았습니다.

구분: RSS 상승이 항상 회수 불가능한 누수라는 뜻은 아닙니다. 정상 시나리오에서는 회수 로그도 별도로 관측됐습니다.

Q12. 직접적인 종료 원인을 무엇으로 판단했나요?

답변의 핵심: 앱이 Memory limit exceeded와 Self-terminating을 기록했습니다. 부팅 실패가 아니라 작업 중 한도 초과에 따른 MemoryGuard 보호 종료로 판단했습니다.

증거: 128MB 로그 (출력 해설), 256MB 로그 (출력 해설).

구분: 128MB를 정확히 찍지 않고 150MB에서 종료한 것은 25MB 단위 증가와 맞습니다. 한도와 최대 RSS를 같은 지표로 취급하지 않습니다.

Q13. 128에서 256으로 올린 조치는 해결인가요?

답변의 핵심: 로그 기준 생존 시간이 17.202→32.363초로 15.161초 증가했지만 다시 같은 정책으로 종료됐습니다. 관측한 문제의 종료를 늦춘 임시 조치입니다.

증거: OOM 비교 보고서.

구분: 임시 조치는 실험자가 한 설정 변경, 보호 정책은 앱의 종료 동작입니다. 소스에서 회수 정책을 수정·검증했다는 뜻은 아닙니다.

Q14. 137이면 커널 OOM이고 143이면 Watchdog인가요?

답변의 핵심: 숫자만으로 발신자나 원인을 확정할 수 없습니다. 137과 143은 시그널 종료 규칙과 일치하지만 로그를 함께 봅니다. CPU After와 Deadlock의 143은 수동 정리 기록이 있습니다.

증거: CPU After 수동 종료 (출력 해설), Deadlock 수동 종료 (출력 해설).

4. CPU 장애

Q15. ps는 1.4%인데 top은 40%여도 되나요?

답변의 핵심: ps는 실행 이후 평균이고 top은 갱신 구간 값이어서 다를 수 있습니다. 앱 내부 Current Load도 별개입니다. 지표 이름·기간을 붙여 설명합니다.

증거: After ps (출력 해설), After top (출력 해설).

Q16. 설정이 80인데 왜 53.11에서 종료됐나요?

답변의 핵심: 정확한 내부 보호 임계치 구현은 확인하지 않았습니다. 환경변수를 OS 사용률 80%의 종료 기준으로 단정하지 않고, 실제 내부 값과 Watchdog 로그를 보고 종료를 판단했습니다.

증거: Before 로그 (출력 해설).

Q17. 왜 0.1초 간격과 첫 화면 제외를 사용했나요?

답변의 핵심: 초기 1초 관측은 짧은 버스트와 종료 직전 구간을 충분히 포착하지 못했습니다. 더 짧게 재수집하고 최초 화면을 이후 갱신 구간과 구분했습니다. 시작·끝·표본 수를 함께 확인했습니다.

증거: 초기 top (출력 해설), 추가 top (출력 해설).

구분: 최대값은 지속 사용률이 아닙니다. 같은 관측법이어도 Before 341개, After 449개로 실제 표본 수가 달랐습니다. 관측하지 않은 시간은 0%로 채우지 않습니다.

Q18. 왜 script를 썼나요?

답변의 핵심: 가상 터미널로 실행해 종료 직전 출력까지 수집하려 했습니다. PTY 재수집에서 Watchdog 문구를 확보했고, -e로 종료 상태를 반영했습니다. 파일 출력과 터미널 출력의 버퍼링 차이가 영향을 줄 수 있습니다.

구분: 버퍼링이 유일한 원인이라고 확정하거나 바이너리에 Python 실행 옵션이 무조건 적용된다고 가정하지 않습니다.

Q19. CPU 설정 변경의 효과와 한계는 무엇인가요?

답변의 핵심: 80→40으로 변경하자 top 관측 최대가 50.0→40.0%가 됐고, 냉각 로그와 55초 시점의 작업 진행을 확인했습니다. 마지막 종료는 수동입니다.

증거: CPU 보고서.

한계: OS 전체 CPU 포화·실제 요청 지연·P99는 측정하지 않았고, 영구적인 종료 회피를 증명한 것도 아닙니다. 장기 기록의 시각 공백을 무시해 연속 가동 시간을 확대하지 않습니다.

5. Deadlock 진단

Q20. PID가 있는데 왜 정상이라고 하지 않았나요?

답변의 핵심: PID 존재는 종료되지 않았다는 뜻입니다. 서로의 락을 기다리는 마지막 로그와, 이후에도 로그 크기·수정 시각이 변하지 않는 정체를 함께 봤습니다.

증거: true 스냅샷1 (출력 해설), true 스냅샷2 (출력 해설).

Q21. 순환 대기를 직접 그려보세요.

답변의 핵심: Thread-1은 Shared_Memory_A를 보유하며 Socket_Pool_B를 기다리고, Thread-2는 B를 보유하며 A를 기다립니다. T1→B→T2→A→T1의 고리가 만들어집니다.

증거: 락 보유·WAITING 로그 (출력 해설).

구분: 두 Worker 이름을 TID 9297·9298 각각에 대응시키는 자료는 없습니다. 어느 TID가 어느 이름인지 확정하지 않습니다.

Q22. 교착의 4대 조건과 futex는 어떻게 설명하나요?

답변의 핵심: 상호 배제, 점유 대기, 비선점, 순환 대기입니다. 이번에는 각 락 보유와 상대 락 요청, 순환 관계가 로그에 있습니다. 비선점은 일반 락 모델에 따른 추론이며 내부 코드를 확인한 것은 아닙니다. futex 대기는 보조 근거이고 단독 진단 기준은 아닙니다.

증거: 조건 대응표, futex 매뉴얼.

Q23. false면 스레드가 하나가 된 건가요?

답변의 핵심: 아닙니다. false에서도 3개 스레드가 관측됐습니다. 로그가 3,474→3,870바이트로 증가하고 작업이 진행된 것을 근거로 이번 교착 경로 회피를 확인했습니다. 메인 futex 대기는 정상 진행 중에도 남았습니다.

증거: false 스냅샷1 (출력 해설), false 스냅샷2 (출력 해설).

구분: 같은 프로세스의 스레드별 RSS 행을 더해 전체 메모리라고 계산하지 않습니다. 증가한 RSS만으로 정상·누수 여부를 판정하지 않습니다.

Q24. 교착을 근본적으로 해결하려면 무엇을 바꿀 수 있나요?

답변의 핵심: 모든 경로에서 락 획득 순서를 통일하고 중첩 락을 줄이는 방안을 제안합니다. 타임아웃·실패 시 롤백·예외 시 해제도 검토 대상입니다. 실제로 수행한 것은 환경변수 비교이며, 비공개 앱을 수정한 것은 아닙니다.

한계: 이번 관측의 회피가 모든 실행 조건의 무교착 보장은 아닙니다. 근본 수정은 소스 접근 후 별도의 검증이 필요합니다.

6. 발표 구성 예시

아래는 원본 증거를 설명하는 구성 예시입니다. 본인의 수행 범위와 새 실험 결과에 맞게 바꿉니다.

도입: "Linux 앱의 메모리·CPU·교착 장애를 재현하고, 앱 로그와 OS 관측을 결합해 분석했습니다. 설정 하나씩을 변경해 비교했고 증거를 실행별 폴더에 보존했습니다."

메모리: "RSS와 내부 메모리 값이 증가했고 MemoryGuard가 한도 초과로 종료했습니다. 128→256으로 올리면 더 오래 실행됐지만 다시 종료해 임시 조치로 분류했습니다. 커널 OOM이나 소스의 누수 위치까지 확인한 것은 아닙니다."

CPU: "짧은 상승을 포착하도록 관제를 보완하고 Watchdog SIGTERM 로그를 확보했습니다. 80→40에서 냉각과 55초 생존을 확인했습니다. 평균 CPU와 구간 CPU, 내부 지표는 구분했습니다."

Deadlock: "PID는 살아 있지만 T1과 T2가 서로의 락을 기다려 작업이 정체됐습니다. false 설정에서는 로그 진행을 확인했습니다. 증거 수집 후의 143은 수동 종료 결과입니다."

마무리: "판단마다 원본 증거를 제시하고, 직접 확인한 사실·추론·미확인 항목을 구분했습니다."

7. 스스로 점검할 기준

수준	점검 기준
다시 읽기	명령을 복사할 수 있지만 왜 쓰는지 설명하기 어려움
기본 이해	명령 목적과 출력 열의 뜻을 설명할 수 있음
증거 해석	원본 파일을 열어 결론과 연결할 수 있음
평가 준비	비교 조건·한계·다른 원인 가능성까지 본인 말로 설명할 수 있음

공식 합격 기준이 아닙니다. 자신이 실제로 설명할 수 있는 수준을 기록합니다. 모르는 구현이나 측정하지 않은 지연량은 "확인하지 않았습니다"라고 답하는 것이 정확합니다.

제4부. 찾아보는 용어집

1. 환경과 파일

용어	쉬운 뜻	이번 과제에서의 의미·확인 방법
운영체제, OS	프로그램에 CPU·메모리·파일 접근을 제공하는 기반	Linux 위에서 앱이 프로세스로 실행됨
Linux	운영체제의 커널과 이를 중심으로 한 환경	프로세스·스레드와 자원을 ps·top으로 관측
커널	운영체제에서 자원 관리를 담당하는 핵심	uname -r로 버전 조회
Ubuntu	Linux를 사용할 수 있게 구성한 배포판	이번 실행 환경, /etc/os-release로 확인
WSL	Windows에서 Linux 환경을 실행하는 기능	PowerShell에서 wsl -d Ubuntu로 접속
터미널	명령과 출력을 주고받는 화면	화면을 닫는 것과 파일을 삭제하는 것은 별개
셸, Bash	입력한 명령을 해석하고 실행하는 프로그램	Ubuntu에서 변수·조건문·반복문 실행
PowerShell	Windows의 명령 실행 환경	PS로 시작하는 프롬프트에서 WSL 시작
프롬프트	명령 입력을 기다린다는 표시	jae0608@HP:~$ 자체는 입력하지 않음
사용자, UID	계정 이름과 숫자 식별 번호	whoami, id -u로 일반 계정 확인
root	관리 권한을 가진 계정	UID 0, 이 앱의 실행 계정으로 사용하지 않음
홈, HOME	사용자의 기본 개인 폴더	~, $HOME, 예전 환경은 /home/jae0608
절대 경로	시작점부터 전체 위치를 적은 경로	/home/jae0608/...
상대 경로	현재 폴더를 기준으로 적은 경로	./agent-leak-app, ../scripts
실행 권한	파일을 프로그램으로 실행할 수 있는 권한	chmod u+x, ls -l의 x
아키텍처	CPU가 처리하는 명령 체계	uname -m, x86_64와 ARM 구분
바이너리	실행할 수 있도록 만들어진 파일	제공 ELF 파일을 실행, 내부 역분석은 하지 않음
ELF	Linux에서 사용하는 실행 파일 등의 형식	file로 확인, 파일 이름만으로 판단하지 않음

2. 셸 문법과 설정

용어·기호	쉬운 뜻	이번 실습의 예
환경변수	프로그램 시작 시 외부에서 전달하는 설정	AGENT_HOME, MEMORY_LIMIT
export	자식 프로그램에도 변수를 전달하도록 지정	export MEMORY_LIMIT=256
source	파일의 명령을 현재 셸에서 실행	env.sh 저장 후 현재 터미널에 적용
$변수	변수에 저장된 값을 꺼내 쓰기	$RUN_DIR는 현재 증거 폴더
큰따옴표	값의 공백 등을 한 인자로 묶으며 변수 확장 허용	"$AGENT_HOME"
작은따옴표	안의 문자열을 대체로 그대로 전달	'agent_api_key_test'
here-document	끝 구분자까지 여러 줄을 입력으로 전달	<<'EOF'와 마지막 EOF
표준 출력	일반적인 결과가 나가는 통로, FD 1	앱의 일반 출력
표준 오류	오류·안내가 나가는 통로, FD 2	monitor.sh의 종료 안내
>	출력을 파일에 저장, 기존 파일 덮어쓰기	> app.log
>>	기존 파일 끝에 출력 추가	>> cleanup.log
2>&1	오류 출력을 현재 표준 출력과 같은 곳에 연결	> app.log 2>&1, 순서가 중요
파이프, |	앞 명령의 표준 출력을 다음 명령에 전달	ps -ef | grep ...
백그라운드, &	셸이 다음 명령을 처리할 수 있게 실행	앱과 관제를 함께 진행
$!	직전 백그라운드 작업 PID	앱 직접 실행과 script 실행에서 대상이 다름
$?	바로 직전 명령의 종료 상태	wait 직후 다른 명령 전에 저장
함수	반복할 명령을 이름으로 묶은 것	같은 스냅샷 조회를 여러 번 실행
CSV	쉼표로 열을 구분한 텍스트 데이터	관제 수치를 표·그래프로 분석
로그	시각과 함께 남긴 동작 기록	app.log, 관제 CSV, cleanup.log

3. 프로세스와 상태

용어	쉬운 뜻	이번 과제의 확인 방법·주의
프로세스	실행 중인 프로그램의 단위	PID가 있어도 작업 진행까지 보장하지 않음
PID	프로세스 식별 번호	ps, 재실행마다 달라지고 재사용 가능
PPID	부모 프로세스의 PID	부모·작업 자식 관계 확인
부모·자식	한 프로세스가 다른 프로세스를 시작한 관계	작업 자식을 관제해야 실제 사용량을 볼 수 있음
스레드	프로세스 안에서 작업하는 실행 흐름	같은 프로세스의 메모리를 공유
TID	스레드 식별 번호	ps -L, Worker 이름과 대응은 별도 증거 필요
R	실행 중이거나 실행 가능한 상태	계속 CPU를 쓴다는 뜻은 아님
S	이벤트 등을 기다리는 대기 상태	정상 프로그램도 자주 관측됨
Z, 좀비	실행은 끝났지만 종료 정보가 남은 상태	monitor.sh는 이 행을 제외
N	낮춘 우선순위 표시	SN의 N, 앱의 nice=10 로그와 부합
nice	CPU 스케줄링 우선순위에 영향을 주는 값	CPU 사용률을 특정 값으로 고정하는 장치는 아님
NLWP, threads	스레드 수	false에서도 실제 3개 관측, 단일 스레드로 단정 금지
WCHAN	커널에서 기다리는 위치의 이름	대기 이유의 단서이지 단독 원인 확정 근거가 아님
futex	사용자 공간 동기화를 지원하는 커널 대기·깨우기 기능	futex_do_wait만으로 교착이라고 하지 않음
시그널	프로세스에 보내는 제어 통지	종료 요청·강제 종료 등
SIGTERM	종료를 요청하는 시그널, 번호 15	Watchdog 또는 수동 정리에 사용, 앱이 처리할 수 있음
SIGKILL	프로세스를 강제 종료하는 시그널, 번호 9	프로세스가 잡아 처리하거나 무시할 수 없음
종료 코드	셸이 받은 실행 결과 번호	137·143만으로 원인·발신자를 확정하지 않음
wait	현재 셸이 시작한 자식 작업의 종료를 기다리고 상태 회수	새 셸에서 과거 종료 상태를 임의 복원할 수 없음
PTY	프로그램에 터미널처럼 보이는 가상 장치	script로 출력 수집, SESSION_PID와 앱 PID 구분

4. 메모리와 CPU

용어	쉬운 뜻	실험과 연결
RAM, 물리 메모리	실행 중인 데이터 등을 담는 실제 메모리	Windows 전체 사용량과 특정 프로세스 값을 구분
가상 주소 공간	프로세스가 사용하는 주소의 논리적 범위	전체 크기가 곧 실제 RAM 점유량은 아님
코드 영역	실행할 명령이 저장되는 부분	메모리 구조 설명의 한 요소
정적 데이터	전역·정적 데이터 등을 저장하는 부분	힙·스택과 역할 구분
Heap, 힙	동적으로 사용하는 데이터 공간	앱의 Current Heap은 내부 기록 값
Stack, 스택	함수 호출의 상태·지역 데이터 등에 쓰는 공간	이번 실험이 스택 오버플로를 입증한 것은 아님
RSS	물리 메모리에 상주하는 프로세스 메모리 양	CSV의 rss_kib, Heap 로그와 별개
VSZ	프로세스 가상 메모리 크기	RSS와 같은 값으로 취급하지 않음
KiB, MiB	각각 1024바이트, 1024KiB	RSS KiB를 1024로 나누면 MiB
MB	일반적으로 100만 바이트, 앱은 자체 MB 표기 사용	앱 내부 계산의 단위 구현은 확인하지 않음
메모리 누수	필요가 끝난 메모리가 회수되지 않아 누적되는 문제	상승만으로 정확한 누수 코드까지 확정하지 않음
캐시	재사용하려고 보관하는 데이터	정상 보관·회수와 해제 누락을 구분해야 함
버퍼	데이터를 잠시 모아두는 공간	종료 직전 출력이 파일에 남는지에 영향 가능
스왑	메모리 페이지를 디스크 쪽에 두는 방식	부족 시 성능 영향 가능, 이번 지연량은 미측정
OOM	사용할 메모리가 부족한 상황	과제 이름과 이번 앱 한도 초과의 직접 원인을 구분
MemoryGuard	이 앱 내부의 메모리 보호 정책	Memory limit exceeded·Self-terminating 로그
OOM Killer	메모리 부족 시 Linux 커널이 프로세스를 선택해 종료하는 기구	이번 기록만으로 발생을 확정하지 않음
CPU	프로그램의 명령을 실행하는 처리 장치	특정 프로세스와 시스템 전체 사용률 구분
CPU 점유율	관측 기간에 CPU를 사용한 비율	분모·기간·표시 모드를 함께 설명
논리 CPU	OS가 스케줄링 대상으로 보는 처리 단위	top 기본 모드의 프로세스 100% 기준에 주의
평균 CPU	실행 이후 전체 기간으로 계산한 사용 비율	ps의 %CPU
구간 CPU	갱신 사이 기간으로 계산한 사용 비율	top의 %CPU
CPU Spike, 버스트	짧은 시간에 부하가 상승하는 현상	긴 관측 간격으로 놓칠 수 있음
Latency, 지연	요청에서 응답까지 걸리는 시간 등	이번에는 별도 요청 지연 벤치마크를 하지 않음
Watchdog	조건을 감시해 보호 동작을 하는 장치·로직	이 앱의 WATCHDOG SIGTERM 로그
냉각, cooldown	부하를 낮추는 동작 구간	40% 도달 후 5%까지 낮아지는 내부 로그

용어	쉬운 뜻	실험과 연결
스케줄링	실행할 작업에 CPU 시간을 배분하는 과정	앱 작업 순서와 OS의 CPU 배분은 다른 층위

5. 락·교착과 보고

용어	쉬운 뜻	이번 실험과 연결
자원	작업이 사용하는 데이터·연결 등의 대상	Shared_Memory_A, Socket_Pool_B
임계 구역	동시에 접근하면 문제가 생길 수 있어 보호하는 작업 부분	락을 보유하고 수행하는 작업
락	공유 자원 접근을 조정하는 잠금	정확한 구현 종류는 확인하지 않음
BLOCKED	필요한 조건·자원 때문에 진행할 수 없는 상태	앱 로그의 표기, ps의 단일 상태 문자와 동일 개념으로 치환하지 않음
Deadlock, 교착	작업들이 서로 기다려 진행하지 못하는 상태	락 순환 관계와 이후 정체로 진단
상호 배제	한 자원을 한 작업만 보유	A와 B의 락 획득
점유 대기	가진 것을 유지하며 다른 것을 기다림	A 보유·B 요청, B 보유·A 요청
비선점	다른 작업의 보유 자원을 강제로 빼앗지 못함	일반 락 모델로 추론, CPU 선점과 구분
순환 대기	기다리는 관계가 고리를 형성	T1→B→T2→A→T1
락 순서 통일	여러 경로에서 같은 순서로 자원을 획득	A 다음 B로 통일하는 근본 조치 제안
식사하는 철학자	여러 작업이 제한된 자원을 나눠 쓰다 서로 기다리는 비유	젓가락 하나씩 가진 채 다른 젓가락을 기다리는 모습
관제, 모니터링	상태를 반복 관측하고 기록	스크립트 수집값과 앱 로그를 결합
표본, 샘플	특정 시점·구간에 수집한 값 하나	1초 간격 사이의 순간 최대는 놓칠 수 있음
관측 최대	수집한 표본 중 가장 큰 값	실제 전체 시간의 절대 최대라고 보장하지 않음
Before & After	변경 전후 비교	변수 하나를 바꾸고 다른 조건은 유지
통제 변수	비교에서 동일하게 유지한 조건	OOM 비교의 CPU와 동시 설정
직접 원인	관측한 현상을 바로 일으킨 원인	MemoryGuard·Watchdog 종료
근본 원인	문제가 생기는 더 깊은 구조·로직의 원인	비공개 코드라 특정 구현은 미확인일 수 있음
임시 조치	당장 영향을 줄이거나 시간을 버는 대응	메모리 한도 상향, 교착 경로 회피
검증	조치 후 실제로 어떤 변화가 있었는지 확인	생존 시간·로그 진행·사용량 비교
증거	다른 사람이 판단을 확인할 수 있는 기록	설정, 시각, PID, CSV, 로그, ps 출력
GitHub Issue 형식	현상·증거·원인·조치를 정리하는 보고 구조	Markdown 문서와 실제 Issues 게시 여부는 구분
추론	확인한 증거로 가능성 있는 설명을 도출	관측 사실 및 미확인 항목과 분리해 표현

6. 반드시 구분할 다섯 쌍

1. **살아 있음 / 진행함:** PID 존재와 새 작업·로그 발생은 다른 확인 항목입니다.
2. **전체 RAM / 앱 한도:** PC Manager의 비율과 MEMORY_LIMIT은 다른 지표입니다.
3. **Heap / RSS:** 앱 내부 기록과 OS 상주 메모리를 같은 수치로 취급하지 않습니다.
4. **보호 정책 / 임시 조치:** 앱의 종료 규칙과 실험자의 설정 변경은 다른 행동입니다.
5. **자동 종료 / 수동 종료:** 같은 143이라도 Watchdog 로그와 cleanup.log로 구분합니다.

참고 문서

- WSL 기본 명령
- ps 매뉴얼
- top 매뉴얼
- futex 매뉴얼
- signal 매뉴얼
- script 매뉴얼
- Python 표준 출력

제5부. 증거 링크별 출력 해설

작성: 2026-10-23. 이 문서는 2026-10-09 원본 증거를 설명합니다. **해설에서 파일 이름을 클릭하면 원본이 열립니다. 브라우저 뒤로가기로 해설로 돌아옵니다.** 빈 파일도 수집 당시 내용이므로 채워 넣지 않았습니다. GitHub가 큰 로그를 접으면 Raw 또는 파일 보기를 사용합니다. 화면의 값은 입력할 명령이 아닙니다.

1. 파일이 만들어지는 이유와 읽는 순서

설정 → 앱 로그 → 관제 값 → 종료/정리 기록 순서로 읽습니다. 설정은 실행 조건이고 관제는 결과입니다. 서로 다른 실행 폴더의 PID·시각을 섞으면 잘못된 결론을 낼 수 있습니다. 파일명에 들어간 시각은 폴더를 만든 시각이며 앱 시작 시각과 반드시 같지는 않습니다.

CSV는 왜 쉼표와 숫자로 보이는가

monitor.sh가 ps 출력을 awk로 변환해 CSV로 저장했기 때문입니다. 각 줄은 한 시점의 표본입니다.

열	뜻	해석할 때 주의할 점
timestamp	측정 시각	+09:00은 한국 시간대이며 경과 시간이 아님
pid / ppid	대상 / 부모 번호	작은 RSS의 실행기와 작업 자식을 구분
elapsed_s	프로세스 경과 초	폴더 생성 이후 시간이 아님
cpu_avg_pct	ps 누적 평균 CPU	top의 짧은 구간 최대와 다를 수 있음
mem_pct	OS가 보는 물리 메모리 비율	Windows PC Manager 비율이나 앱 한도 비율이 아님
rss_kib	상주 메모리, KiB	1024로 나누면 MiB; 마지막 관측값이 순간 절대 최대는 아님
threads	스레드 수	false 설정이라도 전체 스레드가 1개가 되는 것은 아님
state	프로세스 상태	SN은 수면+낮은 우선순위; R은 실행/실행 가능

예를 들어 OOM128 CSV의 962,954,17,0.8,1.8,146376,1,SN은 PID962, 부모954, 경과17초, 누적 평균 CPU0.8%, 물리 메모리 1.8%, RSS146376KiB, 스레드1개, 낮은 우선순위의 수면 상태라는 뜻입니다. CPU가 낮아도 메모리 장애는 발생할 수 있습니다.

로그와 시스템 출력은 왜 모양이 다른가

앱 로그는 앱이 출력한 시각·등급·모듈·문장이고, ps/top은 OS가 관측한 값입니다. [CRITICAL]은 로그 등급, [MemoryGuard]는 기록 주체 이름입니다. >=는 이상 비교입니다. 128MB 실험이 150MB에서 종료되는 것은 로그의 증가가 25MB 단위로 관측됐기 때문입니다.

top의 윗부분 load average는 시스템의 실행/일부 대기 부하 평균이며 퍼센트가 아닙니다. 아래 PID 행의 %CPU를 비교합니다. 첫 프레임은 이후 갱신 구간과 기준이 달라 최대 계산에서 제외했습니다. 짧은 표본의 50%와 앱 내부 53.11%가 다른 것은 곧 오류라는 뜻이 아닙니다.

ps 제목만 남으면 조회 조건에 맞는 프로세스 행이 없는 것입니다. 즉시 성공/실패로 단정하지 않고 조회한 PID와 시각을 확인합니다. futex_do_wait는 동기화 대기 위치입니다. 정상 대기도 가능하므로 로그의 자원 보유·요청과 반복 관측을 함께 읽습니다.

이 구성의 용어 설명

표본은 한 번의 측정, 관측 범위는 실제 기록한 시간 구간, 무결성은 원본 바이트의 보존입니다. 137은 SIGKILL과 일치하는 상태지만 커널 OOM의 단독 증거가 아니고, 143도 Watchdog와 수동 종료 둘 다에서 나올 수 있습니다. 종료 주체는 로그와 함께 판단합니다.

2. OOM 변경 전: 128MB

작업 PID 962. RSS 18,356→146,376KiB로 증가했습니다. 150MB >= 128MB와 Self-terminating 로그가 직접 종료 원인인 MemoryGuard를 설명합니다. 시작 로그부터 종료 요청까지 17.202초입니다.

원본 링크와 파일별 출력 해설

- app.log: 앱의 부팅·작업·보호 로그입니다. PTY 실행이면 script 시작/종료 표지도 포함됩니다. 설정 확인 뒤 마지막 핵심 문장을 읽고 관제 시각과 대조합니다.

- exit-code.txt: wait 직후 저장한 종료 상태입니다. 137/143을 앱 로그·cleanup.log와 대조해 원인과 정리를 구분합니다. 숫자만으로 종료 주체를 알 수 없습니다.

- launcher.pid: 백그라운드로 시작한 실행기의 $! 값입니다. 실제 작업 자식 번호와 다를 수 있으므로 PPID로 연결합니다.

- monitor-954.csv: ps의 시계열 CSV입니다. 위 CSV 열 해설을 보며 RSS·CPU·상태의 시간 변화를 읽습니다. PID가 실행기인지 작업 자식인지 PPID와 함께 확인합니다.

- monitor-954.log: 관제기의 표준 오류·종료 알림입니다. Monitoring ended는 대상 행이 없거나 좀비를 제외해 관제를 끝냈다는 뜻입니다. 정확한 종료 원인은 앱 로그가 설명합니다.

- monitor-962.csv: ps의 시계열 CSV입니다. 위 CSV 열 해설을 보며 RSS·CPU·상태의 시간 변화를 읽습니다. PID가 실행기인지 작업 자식인지 PPID와 함께 확인합니다.

- monitor-962.log: 관제기의 표준 오류·종료 알림입니다. Monitoring ended는 대상 행이 없거나 좀비를 제외해 관제를 끝냈다는 뜻입니다. 정확한 종료 원인은 앱 로그가 설명합니다.

- ps-check.txt: 종료 또는 생존 확인 시점의 ps 결과입니다. 헤더뿐이면 그 조회에 맞는 행이 없다는 뜻이며 로그와 결합해 판단합니다.

- ps-start.txt: 앱 시작 직후 ps 스냅샷입니다. PID/PPID 관계와 RSS를 확인합니다. 경과0초의 CPU 평균은 짧은 초기 구간 값이라 지속 과점유로 단정하지 않습니다.

- settings.txt: 실행 전에 저장한 환경변수입니다. 세 값 중 비교 대상 하나만 달라졌는지 읽습니다. 실제 적용은 부팅 로그와 대조합니다.

- started_at.txt: 실행 명령 직전에 기록한 시각입니다. 부팅 완료 시각과 차이가 있으므로 보고서의 생존 시간은 지정한 시작 로그를 기준으로 계산합니다.

이 구성의 용어 설명

MemoryGuard는 앱 메모리 보호 정책, RSS는 상주량, 생존 시간은 지정한 두 로그 시각의 차이입니다.

3. OOM 변경 후: 256MB

작업 PID 1156. RSS 18,476→274,516KiB로 증가했습니다. 275MB >= 256MB에서 같은 보호 정책으로 종료됐습니다. 32.363초로 15.161초 늘었지만 메모리 증가 자체는 해결되지 않았습니다.

원본 링크와 파일별 출력 해설

- app.log: 앱의 부팅·작업·보호 로그입니다. PTY 실행이면 script 시작/종료 표지도 포함됩니다. 설정 확인 뒤 마지막 핵심 문장을 읽고 관제 시각과 대조합니다.

- exit-code.txt: wait 직후 저장한 종료 상태입니다. 137/143을 앱 로그·cleanup.log와 대조해 원인과 정리를 구분합니다. 숫자만으로 종료 주체를 알 수 없습니다.

- launcher.pid: 백그라운드로 시작한 실행기의 $! 값입니다. 실제 작업 자식 번호와 다를 수 있으므로 PPID로 연결합니다.

- monitor-1148.csv: ps의 시계열 CSV입니다. 위 CSV 열 해설을 보며 RSS·CPU·상태의 시간 변화를 읽습니다. PID가 실행기인지 작업 자식인지 PPID와 함께 확인합니다.

- monitor-1148.log: 관제기의 표준 오류·종료 알림입니다. Monitoring ended는 대상 행이 없거나 좀비를 제외해 관제를 끝냈다는 뜻입니다. 정확한 종료 원인은 앱 로그가 설명합니다.

- monitor-1156.csv: ps의 시계열 CSV입니다. 위 CSV 열 해설을 보며 RSS·CPU·상태의 시간 변화를 읽습니다. PID가 실행기인지 작업 자식인지 PPID와 함께 확인합니다.

- monitor-1156.log: 관제기의 표준 오류·종료 알림입니다. Monitoring ended는 대상 행이 없거나 좀비를 제외해 관제를 끝냈다는 뜻입니다. 정확한 종료 원인은 앱 로그가 설명합니다.

- ps-check.txt: 종료 또는 생존 확인 시점의 ps 결과입니다. 헤더뿐이면 그 조회에 맞는 행이 없다는 뜻이며 로그와 결합해 판단합니다.

- settings.txt: 실행 전에 저장한 환경변수입니다. 세 값 중 비교 대상 하나만 달라졌는지 읽습니다. 실제 적용은 부팅 로그와 대조합니다.

- started_at.txt: 실행 명령 직전에 기록한 시각입니다. 부팅 완료 시각과 차이가 있으므로 보고서의 생존 시간은 지정한 시작 로그를 기준으로 계산합니다.

이 구성의 용어 설명

MemoryGuard는 앱 메모리 보호 정책, RSS는 상주량, 생존 시간은 지정한 두 로그 시각의 차이입니다.

4. CPU 최초 수집: 관제 한계 확인

비교의 주 근거로 삼기 어려운 최초 실행입니다. 1초 top의 30프레임은 종료 전 약 5초를 놓쳤습니다. 관측 최대 4%만 보고 CPU 문제가 없었다고 결론내릴 수 없습니다. 아래 PTY 실행으로 증거를 보완했습니다.

원본 링크와 파일별 출력 해설

- app.log: 앱의 부팅·작업·보호 로그입니다. PTY 실행이면 script 시작/종료 표지도 포함됩니다. 설정 확인 뒤 마지막 핵심 문장을 읽고 관제 시각과 대조합니다.

- exit-code.txt: wait 직후 저장한 종료 상태입니다. 137/143을 앱 로그·cleanup.log와 대조해 원인과 정리를 구분합니다. 숫자만으로 종료 주체를 알 수 없습니다.

- launcher.pid: 백그라운드로 시작한 실행기의 $! 값입니다. 실제 작업 자식 번호와 다를 수 있으므로 PPID로 연결합니다.

- monitor-1522.csv: ps의 시계열 CSV입니다. 위 CSV 열 해설을 보며 RSS·CPU·상태의 시간 변화를 읽습니다. PID가 실행기인지 작업 자식인지 PPID와 함께 확인합니다.

- monitor-1522.log: 관제기의 표준 오류·종료 알림입니다. Monitoring ended는 대상 행이 없거나 좀비를 제외해 관제를 끝냈다는 뜻입니다. 정확한 종료 원인은 앱 로그가 설명합니다.

- monitor-1530.csv: ps의 시계열 CSV입니다. 위 CSV 열 해설을 보며 RSS·CPU·상태의 시간 변화를 읽습니다. PID가 실행기인지 작업 자식인지 PPID와 함께 확인합니다.

- monitor-1530.log: 관제기의 표준 오류·종료 알림입니다. Monitoring ended는 대상 행이 없거나 좀비를 제외해 관제를 끝냈다는 뜻입니다. 정확한 종료 원인은 앱 로그가 설명합니다.

- ps-check.txt: 종료 또는 생존 확인 시점의 ps 결과입니다. 헤더뿐이면 그 조회에 맞는 행이 없다는 뜻이며 로그와 결합해 판단합니다.

- settings.txt: 실행 전에 저장한 환경변수입니다. 세 값 중 비교 대상 하나만 달라졌는지 읽습니다. 실제 적용은 부팅 로그와 대조합니다.

- started_at.txt: 실행 명령 직전에 기록한 시각입니다. 부팅 완료 시각과 차이가 있으므로 보고서의 생존 시간은 지정한 시작 로그를 기준으로 계산합니다.

- top-error.log: top의 표준 오류입니다. 빈 파일은 그 경로로 기록된 오류 문장이 없다는 뜻이며 관제 기간의 충분함까지 보장하지 않습니다. **이 원본은 0바이트입니다.**

- top.log: 반복한 top 화면입니다. 시각 범위와 PID별 %CPU를 읽습니다. 최초 프레임을 제외하며, 마지막 관제 이후 상태는 이 파일만으로 판단하지 않습니다.

이 구성의 용어 설명

Watchdog는 보호 감시, PTY는 가상 터미널, 구간 최대는 기록된 표본 중 최대이며 무한 시간의 최대가 아닙니다.

5. CPU 변경 전: 80 설정

작업 PID 1921. top 최대 50.0%, 앱 내부 Current Load 53.11%와 WATCHDOG/SIGTERM 로그가 있습니다. 서로 다른 시간 범위·계산 기준의 지표입니다. 80 설정을 OS의 정확한 종료 임계치로 단정할 수 없습니다.

원본 링크와 파일별 출력 해설

- app.log: 앱의 부팅·작업·보호 로그입니다. PTY 실행이면 script 시작/종료 표지도 포함됩니다. 설정 확인 뒤 마지막 핵심 문장을 읽고 관제 시각과 대조합니다.

- console.log: script 명령의 표준 출력과 오류를 셸 리다이렉션으로 기록한 파일입니다. app.log와 겹치는 내용이 있어도 독립된 두 번의 재현은 아닙니다.

- exit-code.txt: wait 직후 저장한 종료 상태입니다. 137/143을 앱 로그·cleanup.log와 대조해 원인과 정리를 구분합니다. 숫자만으로 종료 주체를 알 수 없습니다.

- monitor-1920.csv: ps의 시계열 CSV입니다. 위 CSV 열 해설을 보며 RSS·CPU·상태의 시간 변화를 읽습니다. PID가 실행기인지 작업 자식인지 PPID와 함께 확인합니다.

- monitor-1920.log: 관제기의 표준 오류·종료 알림입니다. Monitoring ended는 대상 행이 없거나 좀비를 제외해 관제를 끝냈다는 뜻입니다. 정확한 종료 원인은 앱 로그가 설명합니다.

- monitor-1921.csv: ps의 시계열 CSV입니다. 위 CSV 열 해설을 보며 RSS·CPU·상태의 시간 변화를 읽습니다. PID가 실행기인지 작업 자식인지 PPID와 함께 확인합니다.

- monitor-1921.log: 관제기의 표준 오류·종료 알림입니다. Monitoring ended는 대상 행이 없거나 좀비를 제외해 관제를 끝냈다는 뜻입니다. 정확한 종료 원인은 앱 로그가 설명합니다.

- observed-pids.txt: 관제 대상으로 발견한 앱 PID 목록입니다. 현재 프로세스가 살아 있다는 증거가 아니라 당시 대상 목록입니다.

- ps-check.txt: 종료 또는 생존 확인 시점의 ps 결과입니다. 헤더뿐이면 그 조회에 맞는 행이 없다는 뜻이며 로그와 결합해 판단합니다.

- ps-start.txt: 앱 시작 직후 ps 스냅샷입니다. PID/PPID 관계와 RSS를 확인합니다. 경과0초의 CPU 평균은 짧은 초기 구간 값이라 지속 과점유로 단정하지 않습니다.

- session.pid: script 기록 세션의 PID입니다. 앱 작업 프로세스의 메모리 사용량과 직접 대응시키지 않습니다.

- settings.txt: 실행 전에 저장한 환경변수입니다. 세 값 중 비교 대상 하나만 달라졌는지 읽습니다. 실제 적용은 부팅 로그와 대조합니다.

- started_at.txt: 실행 명령 직전에 기록한 시각입니다. 부팅 완료 시각과 차이가 있으므로 보고서의 생존 시간은 지정한 시작 로그를 기준으로 계산합니다.

- top-error.log: top의 표준 오류입니다. 빈 파일은 그 경로로 기록된 오류 문장이 없다는 뜻이며 관제 기간의 충분함까지 보장하지 않습니다. **이 원본은 0바이트입니다.**

- top.log: 반복한 top 화면입니다. 시각 범위와 PID별 %CPU를 읽습니다. 최초 프레임을 제외하며, 마지막 관제 이후 상태는 이 파일만으로 판단하지 않습니다.

이 구성의 용어 설명

Watchdog는 보호 감시, PTY는 가상 터미널, 구간 최대는 기록된 표본 중 최대이며 무한 시간의 최대가 아닙니다.

6. CPU 변경 후: 40 설정

작업 PID 2359. top 최대 40.0%와 앱 냉각 동작을 관찰했습니다. 55초 확인 시 PID와 로그 진행이 유지됐습니다. 최종 143은 수동 정리와 함께 읽습니다. 전체 서비스 지연 개선을 측정한 실험은 아닙니다.

원본 링크와 파일별 출력 해설

- app.log: 앱의 부팅·작업·보호 로그입니다. PTY 실행이면 script 시작/종료 표지도 포함됩니다. 설정 확인 뒤 마지막 핵심 문장을 읽고 관제 시각과 대조합니다.

- cleanup.log: 실험자가 보낸 manual SIGTERM과 시각·PID 기록입니다. 종료 코드143을 장애 자동 종료로 오해하지 않도록 수집했습니다.

- console.log: script 명령의 표준 출력과 오류를 셸 리다이렉션으로 기록한 파일입니다. app.log와 겹치는 내용이 있어도 독립된 두 번의 재현은 아닙니다.

- exit-code.txt: wait 직후 저장한 종료 상태입니다. 137/143을 앱 로그·cleanup.log와 대조해 원인과 정리를 구분합니다. 숫자만으로 종료 주체를 알 수 없습니다.

- monitor-2358.csv: ps의 시계열 CSV입니다. 위 CSV 열 해설을 보며 RSS·CPU·상태의 시간 변화를 읽습니다. PID가 실행기인지 작업 자식인지 PPID와 함께 확인합니다.

- monitor-2358.log: 관제기의 표준 오류·종료 알림입니다. Monitoring ended는 대상 행이 없거나 좀비를 제외해 관제를 끝냈다는 뜻입니다. 정확한 종료 원인은 앱 로그가 설명합니다.

- monitor-2359.csv: ps의 시계열 CSV입니다. 위 CSV 열 해설을 보며 RSS·CPU·상태의 시간 변화를 읽습니다. PID가 실행기인지 작업 자식인지 PPID와 함께 확인합니다.

- monitor-2359.log: 관제기의 표준 오류·종료 알림입니다. Monitoring ended는 대상 행이 없거나 좀비를 제외해 관제를 끝냈다는 뜻입니다. 정확한 종료 원인은 앱 로그가 설명합니다.

- observed-pids.txt: 관제 대상으로 발견한 앱 PID 목록입니다. 현재 프로세스가 살아 있다는 증거가 아니라 당시 대상 목록입니다.

- observed_at.txt: 확인 명령을 실행한 시각입니다. 앱 마지막 로그 시각과 달라도 기록 오류가 아니라 확인이 늦었을 수 있습니다.

- ps-after-cleanup.txt: 수동 종료 후 ps 결과입니다. 헤더만 남는 것은 당시 대상 프로세스가 조회되지 않았다는 정리 확인입니다.

- ps-check.txt: 종료 또는 생존 확인 시점의 ps 결과입니다. 헤더뿐이면 그 조회에 맞는 행이 없다는 뜻이며 로그와 결합해 판단합니다.

- ps-start.txt: 앱 시작 직후 ps 스냅샷입니다. PID/PPID 관계와 RSS를 확인합니다. 경과0초의 CPU 평균은 짧은 초기 구간 값이라 지속 과점유로 단정하지 않습니다.

- session.pid: script 기록 세션의 PID입니다. 앱 작업 프로세스의 메모리 사용량과 직접 대응시키지 않습니다.

- settings.txt: 실행 전에 저장한 환경변수입니다. 세 값 중 비교 대상 하나만 달라졌는지 읽습니다. 실제 적용은 부팅 로그와 대조합니다.

- started_at.txt: 실행 명령 직전에 기록한 시각입니다. 부팅 완료 시각과 차이가 있으므로 보고서의 생존 시간은 지정한 시작 로그를 기준으로 계산합니다.

- top-error.log: top의 표준 오류입니다. 빈 파일은 그 경로로 기록된 오류 문장이 없다는 뜻이며 관제 기간의 충분함까지 보장하지 않습니다. **이 원본은 0바이트입니다.**

- top.log: 반복한 top 화면입니다. 시각 범위와 PID별 %CPU를 읽습니다. 최초 프레임을 제외하며, 마지막 관제 이후 상태는 이 파일만으로 판단하지 않습니다.

이 구성의 용어 설명

Watchdog는 보호 감시, PTY는 가상 터미널, 구간 최대는 기록된 표본 중 최대이며 무한 시간의 최대가 아닙니다.

7. Deadlock 변경 전: true

작업 PID 9226. 두 스냅샷에서 RSS 18,512KiB, 로그 2,792바이트가 유지됐습니다. T1은 A를 가진 채 B를, T2는 B를 가진 채 A를 기다리는 로그가 순환 대기를 보여줍니다. 살아 있으나 작업이 정체된 상태입니다.

원본 링크와 파일별 출력 해설

- app.log: 앱의 부팅·작업·보호 로그입니다. PTY 실행이면 script 시작/종료 표지도 포함됩니다. 설정 확인 뒤 마지막 핵심 문장을 읽고 관제 시각과 대조합니다.

- cleanup.log: 실험자가 보낸 manual SIGTERM과 시각·PID 기록입니다. 종료 코드143을 장애 자동 종료로 오해하지 않도록 수집했습니다.

- console.log: script 명령의 표준 출력과 오류를 셸 리다이렉션으로 기록한 파일입니다. app.log와 겹치는 내용이 있어도 독립된 두 번의 재현은 아닙니다.

- exit-code.txt: wait 직후 저장한 종료 상태입니다. 137/143을 앱 로그·cleanup.log와 대조해 원인과 정리를 구분합니다. 숫자만으로 종료 주체를 알 수 없습니다.

- monitor-9225.csv: ps의 시계열 CSV입니다. 위 CSV 열 해설을 보며 RSS·CPU·상태의 시간 변화를 읽습니다. PID가 실행기인지 작업 자식인지 PPID와 함께 확인합니다.

- monitor-9225.log: 관제기의 표준 오류·종료 알림입니다. Monitoring ended는 대상 행이 없거나 좀비를 제외해 관제를 끝냈다는 뜻입니다. 정확한 종료 원인은 앱 로그가 설명합니다.

- monitor-9226.csv: ps의 시계열 CSV입니다. 위 CSV 열 해설을 보며 RSS·CPU·상태의 시간 변화를 읽습니다. PID가 실행기인지 작업 자식인지 PPID와 함께 확인합니다.

- monitor-9226.log: 관제기의 표준 오류·종료 알림입니다. Monitoring ended는 대상 행이 없거나 좀비를 제외해 관제를 끝냈다는 뜻입니다. 정확한 종료 원인은 앱 로그가 설명합니다.

- observed-pids.txt: 관제 대상으로 발견한 앱 PID 목록입니다. 현재 프로세스가 살아 있다는 증거가 아니라 당시 대상 목록입니다.

- ps-after-cleanup.txt: 수동 종료 후 ps 결과입니다. 헤더만 남는 것은 당시 대상 프로세스가 조회되지 않았다는 정리 확인입니다.

- session.pid: script 기록 세션의 PID입니다. 앱 작업 프로세스의 메모리 사용량과 직접 대응시키지 않습니다.

- settings.txt: 실행 전에 저장한 환경변수입니다. 세 값 중 비교 대상 하나만 달라졌는지 읽습니다. 실제 적용은 부팅 로그와 대조합니다.

- snapshot-1.txt: 같은 명령 묶음으로 찍은 상태 사진입니다. ps -ef의 존재, ps -L의 TID·TIME·WCHAN, stat의 로그 바이트·수정 시각, 마지막 앱 문장을 읽습니다. 두 스냅샷의 차이와 로그의 락 관계로 정체/진행을 판단합니다.

- snapshot-2.txt: 같은 명령 묶음으로 찍은 상태 사진입니다. ps -ef의 존재, ps -L의 TID·TIME·WCHAN, stat의 로그 바이트·수정 시각, 마지막 앱 문장을 읽습니다. 두 스냅샷의 차이와 로그의 락 관계로 정체/진행을 판단합니다.

- started_at.txt: 실행 명령 직전에 기록한 시각입니다. 부팅 완료 시각과 차이가 있으므로 보고서의 생존 시간은 지정한 시작 로그를 기준으로 계산합니다.

- worker.pid: 실행 당시 수집한 보조 기록입니다. 파일의 명령·시각·대상을 확인하고 같은 폴더의 설정 및 앱 로그와 대조합니다.

이 구성의 용어 설명

BLOCKED는 앱 로그의 대기 표현, 락은 공유 자원 접근 제어, 순환 대기는 서로의 자원을 기다리는 고리입니다.

8. Deadlock 변경 후: false

작업 PID 11294. 10초 간격 두 스냅샷에서 RSS 121,000→197,812KiB, 로그 3,474→3,870바이트로 변했고 작업 로그도 진행했습니다. 관측 구간에서 교착 경로가 회피됐다는 뜻입니다. 스레드 3개가 남는 것은 false가 전체 단일 스레드 설정이라는 뜻이 아니기 때문입니다.

원본 링크와 파일별 출력 해설

- app.log: 앱의 부팅·작업·보호 로그입니다. PTY 실행이면 script 시작/종료 표지도 포함됩니다. 설정 확인 뒤 마지막 핵심 문장을 읽고 관제 시각과 대조합니다.

- cleanup.log: 실험자가 보낸 manual SIGTERM과 시각·PID 기록입니다. 종료 코드143을 장애 자동 종료로 오해하지 않도록 수집했습니다.

- console.log: script 명령의 표준 출력과 오류를 셸 리다이렉션으로 기록한 파일입니다. app.log와 겹치는 내용이 있어도 독립된 두 번의 재현은 아닙니다.

- exit-code.txt: wait 직후 저장한 종료 상태입니다. 137/143을 앱 로그·cleanup.log와 대조해 원인과 정리를 구분합니다. 숫자만으로 종료 주체를 알 수 없습니다.

- monitor-11293.csv: ps의 시계열 CSV입니다. 위 CSV 열 해설을 보며 RSS·CPU·상태의 시간 변화를 읽습니다. PID가 실행기인지 작업 자식인지 PPID와 함께 확인합니다.

- monitor-11293.log: 관제기의 표준 오류·종료 알림입니다. Monitoring ended는 대상 행이 없거나 좀비를 제외해 관제를 끝냈다는 뜻입니다. 정확한 종료 원인은 앱 로그가 설명합니다.

- monitor-11294.csv: ps의 시계열 CSV입니다. 위 CSV 열 해설을 보며 RSS·CPU·상태의 시간 변화를 읽습니다. PID가 실행기인지 작업 자식인지 PPID와 함께 확인합니다.

- monitor-11294.log: 관제기의 표준 오류·종료 알림입니다. Monitoring ended는 대상 행이 없거나 좀비를 제외해 관제를 끝냈다는 뜻입니다. 정확한 종료 원인은 앱 로그가 설명합니다.

- observed-pids.txt: 관제 대상으로 발견한 앱 PID 목록입니다. 현재 프로세스가 살아 있다는 증거가 아니라 당시 대상 목록입니다.

- ps-after-cleanup.txt: 수동 종료 후 ps 결과입니다. 헤더만 남는 것은 당시 대상 프로세스가 조회되지 않았다는 정리 확인입니다.

- session.pid: script 기록 세션의 PID입니다. 앱 작업 프로세스의 메모리 사용량과 직접 대응시키지 않습니다.

- settings.txt: 실행 전에 저장한 환경변수입니다. 세 값 중 비교 대상 하나만 달라졌는지 읽습니다. 실제 적용은 부팅 로그와 대조합니다.

- snapshot-1.txt: 같은 명령 묶음으로 찍은 상태 사진입니다. ps -ef의 존재, ps -L의 TID·TIME·WCHAN, stat의 로그 바이트·수정 시각, 마지막 앱 문장을 읽습니다. 두 스냅샷의 차이와 로그의 락 관계로 정체/진행을 판단합니다.

- snapshot-2.txt: 같은 명령 묶음으로 찍은 상태 사진입니다. ps -ef의 존재, ps -L의 TID·TIME·WCHAN, stat의 로그 바이트·수정 시각, 마지막 앱 문장을 읽습니다. 두 스냅샷의 차이와 로그의 락 관계로 정체/진행을 판단합니다.

- started_at.txt: 실행 명령 직전에 기록한 시각입니다. 부팅 완료 시각과 차이가 있으므로 보고서의 생존 시간은 지정한 시작 로그를 기준으로 계산합니다.

- worker.pid: 실행 당시 수집한 보조 기록입니다. 파일의 명령·시각·대상을 확인하고 같은 폴더의 설정 및 앱 로그와 대조합니다.

이 구성의 용어 설명

BLOCKED는 앱 로그의 대기 표현, 락은 공유 자원 접근 제어, 순환 대기는 서로의 자원을 기다리는 고리입니다.

9. 그래프·환경·검증 파일 해설

- 메모리 그래프: 가로축은 경과 시간, 세로축은 RSS입니다. 선이 끝난 위치는 관측 종료이며 커널 OOM을 표시한 선이 아닙니다. 메모리 한도는 앱 내부 기준과 구분합니다.

- CPU 그래프: top의 구간별 프로세스 CPU를 보여줍니다. 80과 40은 실행 설정 이름이며 전체 노트북 CPU 사용률이 아닙니다.

- 교착 비교 그래프: 두 시점의 로그·메모리 변화 비교입니다. false에서 값이 증가해도 그 자체로 누수라는 뜻은 아니며 작업 진행 로그와 함께 읽습니다.

- 환경 정보: uname은 커널, id는 계정, os-release는 배포판, nproc는 사용 가능한 처리기 수, free는 메모리 상태를 기록한 것입니다. free의 buff/cache는 모두 즉시 회수 불가능한 누수 메모리가 아닙니다.

- 무결성 목록: SHA-256 값과 파일 경로 쌍입니다. 같은 바이트인지 비교하는 자료이며 기술적 결론의 옳고 그름을 판정하는 값은 아닙니다.

- 측정 요약 JSON: 원본에서 정리한 구조화 데이터입니다. 중괄호는 객체, 대괄호는 목록이며 키 옆 값의 단위를 확인합니다. 새로운 실행 로그가 아닙니다.

평가에서는 원본을 연 뒤 "무엇을 수집했다 → 왜 이런 형식인가 → 어떤 값이 어떻게 바뀌었다 → 어느 로그와 연결된다 → 무엇은 미확인이다" 순서로 설명합니다.